

INFORMACIÓN GENERAL

Fecha de creación	17-07-2025	Dirección:	DIRECCIÓN DE OPERACIONES
-------------------	------------	------------	--------------------------

Escuela	INGENIERIAS
Programa	INGENIERIA DE SISTEMAS

IDENTIFICACIÓN DEL CURSO

Malla:	VG002
Código:	ISP01
Asignatura:	PROGRAMACION AVANZADA
Semestre (Nivel)	4
Temática:	1

Tema: Programación Avanzada y Desarrollo de Software Ágil

1. Definición

La programación avanzada se refiere al conjunto de técnicas, patrones, paradigmas y prácticas que permiten desarrollar aplicaciones complejas, eficientes y escalables, integrando conocimientos más allá de los fundamentos básicos de la programación. En paralelo, el desarrollo de software ágil es un enfoque iterativo e incremental que prioriza la entrega temprana de valor al cliente, la adaptabilidad a los cambios y la colaboración entre equipos.

Combinar programación avanzada con metodologías ágiles permite al desarrollador escribir código de calidad alineado con ciclos de entrega cortos, iterativos y centrados en las necesidades del cliente, manteniendo la sostenibilidad del desarrollo a largo plazo.

2. Conceptos clave

- Principios de DevOps en desarrollo de software: DevOps es una filosofía que integra desarrollo (Dev) y operaciones (Ops) para acelerar la entrega de software, mejorar la calidad y automatizar procesos de integración y despliegue continuo. El programador avanzado incorpora prácticas como

versionamiento, testing automatizado y despliegue continuo desde la primera etapa del desarrollo.

- Buenas prácticas para documentación y gestión del ciclo de vida: La documentación clara del código, el uso de comentarios útiles, el control de versiones (por ejemplo, Git) y la aplicación de herramientas de gestión de incidencias permiten mantener un desarrollo organizado y comprensible para equipos distribuidos.
- Metodologías ágiles: Scrum y Kanban: Scrum estructura el desarrollo en sprints, revisiones y retrospectivas para mejorar continuamente el producto. Kanban, por su parte, utiliza un tablero visual para gestionar y optimizar el flujo de tareas, permitiendo detectar cuellos de botella y mejorar la eficiencia.

3. Ejemplos en texto

- Ejemplo 1: Aplicación de DevOps en un proyecto .NET: Un equipo que desarrolla una API en C# utiliza Git para versionar el código, Azure DevOps para integrar pruebas automatizadas y desplegar cada nueva versión en un entorno de pruebas. Fragmento de un archivo .yaml para definir integración continua en Azure DevOps:

trigger:

- main

pool:

vmImage: 'windows-latest'

steps:

- task: UseDotNet@2

inputs:

packageType: 'sdk'

version: '6.0.x'

- script: dotnet build

displayName: 'Build project'

- script: dotnet test

displayName: 'Run tests'

- Ejemplo 2: Documentación de un método en C#: Documentar correctamente las funciones ayuda a otros desarrolladores a entender el propósito y la firma de un método:

```
/// <summary>
```

```
/// Calcula el salario neto de un empleado.
```

```
/// </summary>
```

```
/// <param name="salarioBruto">Salario bruto mensual.</param>
```

```
/// <param name="descuentos">Descuentos aplicables.</param>
```

```
/// <returns>Salario neto.</returns>
```

```
public decimal CalcularSalarioNeto(decimal salarioBruto, decimal  
descuentos)
```

```
{
```

```
    return salarioBruto - descuentos;
```

```
}
```

- Ejemplo 3: Uso de Scrum para gestionar un proyecto de sistema escolar: El equipo define un backlog de historias de usuario (por ejemplo: “Como profesor quiero registrar notas para mis estudiantes”) y las distribuye en sprints de dos semanas. Cada tarea se representa en un tablero Kanban, moviéndose de “Pendiente” a “En progreso” y finalmente a “Terminado”.

4. Conclusión

El dominio de la programación avanzada no solo implica conocer estructuras y sintaxis más complejas, sino también adoptar enfoques ágiles y disciplinados para entregar software de calidad que responda a las necesidades del usuario final. Al incorporar principios de DevOps, metodologías ágiles y buenas prácticas de documentación, los ingenieros pueden construir soluciones robustas, mantener la flexibilidad del proyecto y reducir los tiempos de entrega.

5. Referencias (APA 7)

- Hernández Bejarno, M. (2020). Ciclo de vida de desarrollo ágil de software seguro. Fundación Universitaria Los Libertadores. <https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/197008>
- Simpson, D. (2023). Using Trello to deliver Kanban style library staff training. Mississippi Libraries, 86(3), 44–49.
- MARK J. PRICE. (2021). C# 10 and .NET 6 – Modern Cross-Platform Development: Build Apps, Websites, and Services with ASP.NET Core 6, Blazor, and EF Core 6 Using Visual Studio 2022 and Visual Studio Code. Packt Publishing.
- Vegas Gertrudix, J. M. (2022). Java 17: programación avanzada. RA-MA.





Programación Avanzada y Desarrollo de Software Ágil

La programación avanzada combina técnicas, patrones y paradigmas que permiten desarrollar aplicaciones complejas, eficientes y escalables. Cuando se integra con metodologías ágiles, permite a los desarrolladores escribir código de calidad alineado con ciclos de entrega cortos, iterativos y centrados en las necesidades del cliente, manteniendo la sostenibilidad del desarrollo a largo plazo.

Definición y Fundamentos

Programación Avanzada

Se refiere al conjunto de técnicas, patrones, paradigmas y prácticas que permiten desarrollar aplicaciones complejas, eficientes y escalables, integrando conocimientos más allá de los fundamentos básicos de la programación.

Estas técnicas avanzadas permiten crear soluciones robustas que pueden adaptarse a entornos cambiantes y requisitos complejos, manteniendo la calidad del código y facilitando su mantenimiento a largo plazo.

La combinación de programación avanzada con metodologías ágiles crea un entorno de desarrollo donde la excelencia técnica se alinea con procesos flexibles orientados al valor, permitiendo entregar soluciones de alta calidad que realmente satisfacen las necesidades del usuario final.

Desarrollo de Software Ágil

Es un enfoque iterativo e incremental que prioriza la entrega temprana de valor al cliente, la adaptabilidad a los cambios y la colaboración entre equipos.

Las metodologías ágiles permiten responder rápidamente a los cambios en los requisitos, mantener una comunicación constante con los stakeholders y entregar software funcional en ciclos cortos, lo que reduce el riesgo y aumenta la satisfacción del cliente.

Principios de DevOps en Desarrollo de Software

DevOps es una filosofía que integra desarrollo (Dev) y operaciones (Ops) para acelerar la entrega de software, mejorar la calidad y automatizar procesos de integración y despliegue continuo. El programador avanzado incorpora prácticas como versionamiento, testing automatizado y despliegue continuo desde la primera etapa del desarrollo.

Integración Continua

Permite a los desarrolladores integrar su código en un repositorio compartido varias veces al día, verificando automáticamente la calidad del código mediante pruebas automatizadas.

Entrega Continua

Garantiza que el código puede ser desplegado en cualquier momento en entornos de producción, reduciendo el tiempo entre la implementación de una característica y su disponibilidad para los usuarios.

Infraestructura como Código

Permite gestionar y aprovisionar la infraestructura mediante definiciones de código, facilitando la creación de entornos consistentes y reproducibles.

```
trigger:
- main

pool:
  vmImage: 'windows-latest'

steps:
- task: UseDotNet@2
  inputs:
    packageType: 'sdk'
    version: '6.0.x'
- script: dotnet build
  displayName: 'Build project'
- script: dotnet test
  displayName: 'Run tests'
```

Este ejemplo muestra un archivo YAML para definir integración continua en Azure DevOps, automatizando la compilación y pruebas de un proyecto .NET cada vez que se realiza un cambio en la rama principal.

Buenas Prácticas para Documentación y Gestión del Ciclo de Vida

La documentación clara del código, el uso de comentarios útiles, el control de versiones (por ejemplo, Git) y la aplicación de herramientas de gestión de incidencias permiten mantener un desarrollo organizado y comprensible para equipos distribuidos.

Elementos clave:

- Documentación de código con comentarios descriptivos
- Control de versiones con sistemas como Git
- Gestión de incidencias y seguimiento de errores
- Revisiones de código entre pares
- Estándares de codificación consistentes

```
///  
/// Calcula el salario neto de un empleado.  
///  
/// Salario bruto mensual.  
/// Descuentos aplicables.  
/// Salario neto.  
public decimal CalcularSalarioNeto(decimal salarioBruto, decimal  
descuentos)  
{  
    return salarioBruto - descuentos;  
}
```

Este ejemplo muestra la documentación adecuada de un método en C#, facilitando que otros desarrolladores entiendan rápidamente su propósito, parámetros y valor de retorno.

Metodologías Ágiles: Scrum y Kanban



Product Backlog

Lista priorizada de todas las funcionalidades, mejoras y correcciones que deben implementarse en el producto.



Sprint Planning

Reunión donde el equipo selecciona elementos del Product Backlog para completar durante el Sprint.



Sprint (2-4 semanas)

Período de tiempo fijo durante el cual se desarrolla un incremento del producto potencialmente entregable.



Sprint Review

Demostración del incremento completado y obtención de feedback de los stakeholders.



Sprint Retrospective

Análisis del proceso para identificar mejoras para el siguiente Sprint.

Scrum

Estructura el desarrollo en sprints, revisiones y retrospectivas para mejorar continuamente el producto. Define roles específicos (Scrum Master, Product Owner, Equipo de Desarrollo) y ceremonias que facilitan la comunicación y la adaptación.

Kanban

Utiliza un tablero visual para gestionar y optimizar el flujo de tareas, permitiendo detectar cuellos de botella y mejorar la eficiencia. Se centra en limitar el trabajo en progreso y optimizar el flujo de entrega continua.

Ejemplo: Un equipo que desarrolla un sistema escolar define un backlog de historias de usuario (por ejemplo: "Como profesor quiero registrar notas para mis estudiantes") y las distribuye en sprints de dos semanas. Cada tarea se representa en un tablero Kanban, moviéndose de "Pendiente" a "En progreso" y finalmente a "Terminado".

Caso Práctico: Sistema Escolar con Metodologías Ágiles

Sprint 1: Gestión de Usuarios

1

Historias de usuario:

- Como administrador quiero registrar nuevos profesores
- Como administrador quiero registrar nuevos estudiantes
- Como usuario quiero iniciar sesión con mis credenciales

Sprint 3: Evaluaciones

3

Historias de usuario:

- Como profesor quiero registrar notas para mis estudiantes
- Como profesor quiero crear evaluaciones con diferentes tipos de preguntas
- Como estudiante quiero ver mis calificaciones

Sprint 2: Gestión Académica

2

Historias de usuario:

- Como profesor quiero crear nuevos cursos
- Como profesor quiero asignar estudiantes a mis cursos
- Como estudiante quiero ver mis cursos asignados

En este caso práctico, el equipo de desarrollo implementa Scrum para estructurar el trabajo en sprints de dos semanas, con reuniones diarias para sincronizar el avance. Utilizan un tablero Kanban para visualizar el flujo de trabajo, limitando el número de tareas en progreso para evitar la sobrecarga. Cada historia de usuario se descompone en tareas técnicas que se estiman y asignan durante la planificación del sprint.

Al final de cada sprint, se realiza una demostración del incremento del producto a los stakeholders (directivos escolares y representantes de profesores) para obtener feedback temprano y realizar ajustes si es necesario. La retrospectiva permite al equipo mejorar continuamente su proceso de desarrollo.

Integración de Programación Avanzada y Metodologías Ágiles

Beneficios de la Integración

Mayor Calidad del Software

Las prácticas de programación avanzada como TDD, patrones de diseño y refactorización continua, combinadas con la entrega iterativa, permiten detectar y corregir problemas tempranamente.

Adaptabilidad a Cambios

El código bien estructurado facilita la implementación de cambios requeridos por el enfoque ágil, reduciendo la deuda técnica y permitiendo pivotes rápidos.

Entrega Continua de Valor

La automatización de pruebas y despliegues permite entregar nuevas funcionalidades con mayor frecuencia y confiabilidad, alineándose con los principios ágiles.

La integración efectiva de programación avanzada y metodologías ágiles requiere un equilibrio entre la excelencia técnica y la entrega de valor al cliente. Los equipos más exitosos mantienen un enfoque disciplinado en ambos aspectos, reconociendo que el código de calidad facilita la agilidad a largo plazo, mientras que los procesos ágiles proporcionan el marco para entregar ese código de manera eficiente y alineada con las necesidades del negocio.

Desafíos y Soluciones

Desafío	Solución
Equilibrar calidad técnica y velocidad de entrega	Incorporar prácticas de XP como programación en parejas y TDD
Mantener la arquitectura flexible	Aplicar principios SOLID y refactorización continua
Gestionar la complejidad creciente	Utilizar patrones de diseño y arquitectura modular
Asegurar la calidad en ciclos cortos	Implementar pruebas automatizadas y revisiones de código

Conclusiones y Referencias

Conclusiones

El dominio de la programación avanzada no solo implica conocer estructuras y sintaxis más complejas, sino también adoptar enfoques ágiles y disciplinados para entregar software de calidad que responda a las necesidades del usuario final. Al incorporar principios de DevOps, metodologías ágiles y buenas prácticas de documentación, los ingenieros pueden construir soluciones robustas, mantener la flexibilidad del proyecto y reducir los tiempos de entrega.

La combinación de estas disciplinas permite a los equipos de desarrollo:

- Responder rápidamente a los cambios del mercado
- Mantener un alto estándar de calidad técnica
- Reducir el tiempo desde la idea hasta la implementación
- Mejorar la colaboración entre equipos multidisciplinarios
- Crear software que realmente satisface las necesidades de los usuarios

Referencias (APA 7)

- Hernández Bejarno, M. (2020). Ciclo de vida de desarrollo ágil de software seguro. Fundación Universitaria Los Libertadores.
<https://elibronet.zproxy.cun.edu.co/es/lc/bibliocun/titulos/197008>
- Simpson, D. (2023). Using Trello to deliver Kanban style library staff training. *Mississippi Libraries*, 86(3), 44–49.
- MARK J. PRICE. (2021). C# 10 and .NET 6 Modern Cross-Platform Development: Build Apps, Websites, and Services with ASP.NET Core 6, Blazor, and EF Core 6 Using Visual Studio 2022 and Visual Studio Code. Packt Publishing.
- Vegas Gertrudix, J. M. (2022). Java 17: programación avanzada. RA-MA.

INFORMACIÓN GENERAL

Fecha de creación	17-07-2025	Dirección:	DIRECCIÓN DE OPERACIONES
-------------------	------------	------------	--------------------------

Escuela	INGENIERIAS
Programa	INGENIERIA DE SISTEMAS

IDENTIFICACIÓN DEL CURSO

Malla:	VG002
Código:	ISP01
Asignatura:	PROGRAMACION AVANZADA
Semestre (Nivel)	4
Temática:	2

Tema: Introducción a la Programación Avanzada

1. Definición

La programación avanzada se construye sobre los fundamentos de la programación básica, incorporando conceptos, técnicas y herramientas que permiten al desarrollador escribir código más eficiente, reutilizable y orientado a problemas reales complejos. Esta unidad refuerza conceptos clave de la programación orientada a objetos, el uso adecuado de entornos de desarrollo integrados (IDE), la configuración de proyectos en lenguajes como C# o Visual Basic, y las prácticas iniciales de documentación del código.

Dominar estos aspectos es esencial para escribir aplicaciones modulares, mantenibles y alineadas con estándares profesionales.

2. Conceptos clave

- Repaso de conceptos clave de POO: La programación orientada a objetos (POO) organiza el software en torno a clases que encapsulan atributos y métodos, permitiendo reutilización y extensión mediante mecanismos como la herencia y el polimorfismo.

- Manejo de entornos de desarrollo integrados (IDE): Un IDE como Visual Studio proporciona un entorno completo para escribir, depurar y administrar proyectos de software, facilitando tareas como la gestión de dependencias, el control de versiones y la ejecución de pruebas.
 - Configuración de proyectos básicos: Inicia la creación de un proyecto, define el lenguaje de programación y establece la estructura de carpetas, archivos y bibliotecas necesarias para el desarrollo.
 - Introducción a la documentación de código: La documentación mejora la mantenibilidad, facilita el trabajo en equipo y asegura que las decisiones de diseño sean comprensibles para otros desarrolladores y para el futuro.
-

3. Ejemplos en texto

- Ejemplo 1: Clase y herencia en C#: Definición de una clase base Animal y una subclase Perro que hereda atributos y métodos:

```
public class Animal
{
    public string Nombre { get; set; }

    public void Comer()
    {
        Console.WriteLine($"{Nombre} está comiendo.");
    }
}

public class Perro : Animal
{
    public void Ladrar()
    {
        Console.WriteLine($"{Nombre} está ladrando.");
    }
}

// Uso:

Perro miPerro = new Perro { Nombre = "Fido" };

miPerro.Comer();
```

```
miPerro.Ladrar();
```

- Ejemplo 2: Documentación XML en métodos

```
/// <summary>
```

```
/// Calcula el área de un rectángulo.
```

```
/// </summary>
```

```
/// <param name="ancho">Ancho del rectángulo.</param>
```

```
/// <param name="alto">Alto del rectángulo.</param>
```

```
/// <returns>Área calculada.</returns>
```

```
public double CalcularArea(double ancho, double alto)
```

```
{
```

```
    return ancho * alto;
```

```
}
```

4. Conclusión

Esta unidad refuerza los fundamentos sobre los cuales se desarrolla la programación avanzada, destacando la importancia de escribir código modular y legible, configurar correctamente los proyectos y utilizar las herramientas profesionales disponibles. Estos conocimientos preparan a los estudiantes para abordar retos más complejos en programación, garantizando calidad y profesionalismo en sus soluciones.

5. Referencias (APA 7)

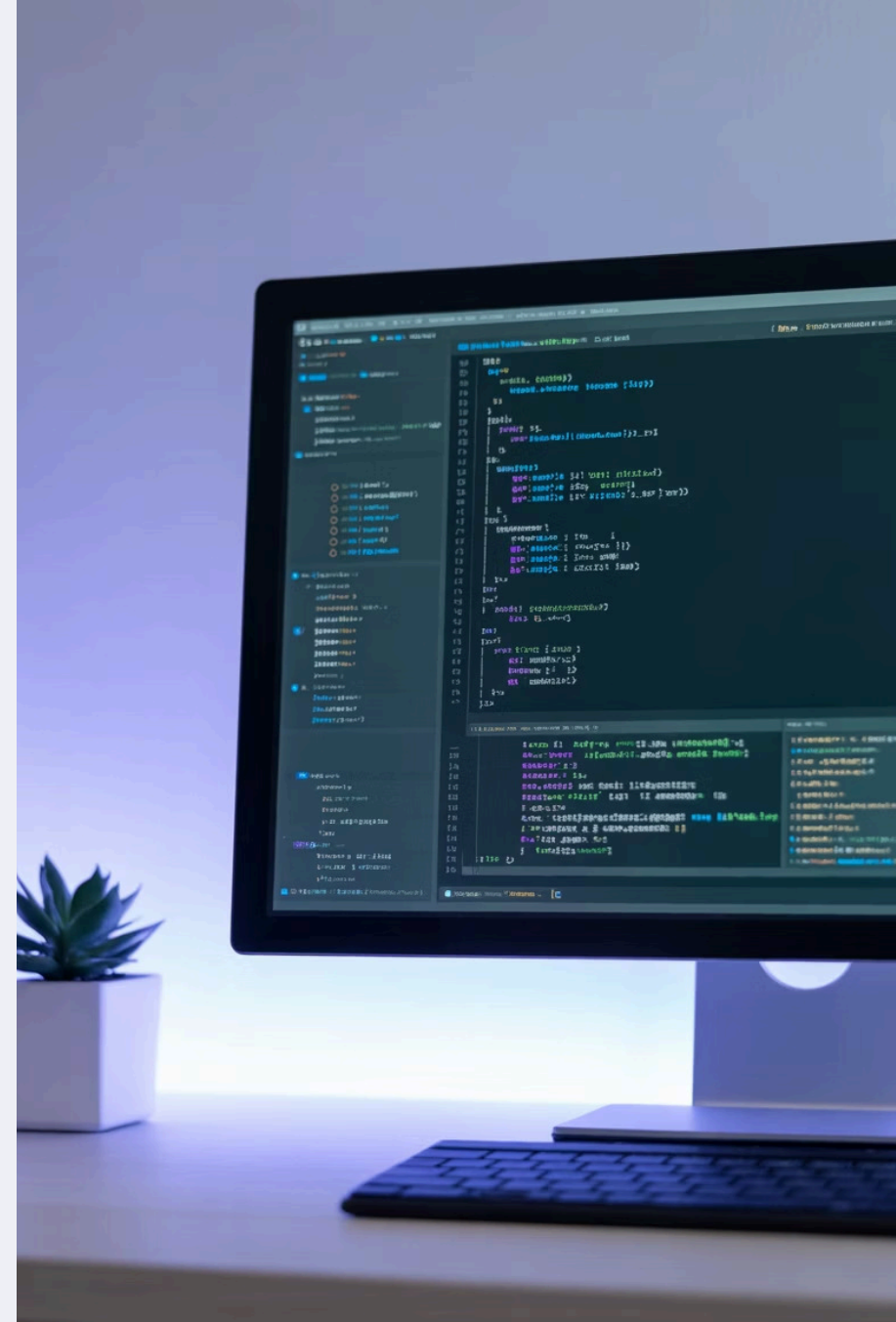
- MARK J. PRICE. (2021). C# 10 and .NET 6 – Modern Cross-Platform Development: Build Apps, Websites, and Services with ASP.NET Core 6, Blazor, and EF Core 6 Using Visual Studio 2022 and Visual Studio Code. Packt Publishing.
- Vegas Gertrudix, J. M. (2022). Java 17: programación avanzada. RA-MA.
- Beiroa Mosquera, R. (2018). Aprender Arduino, prototipado y programación avanzada con 100 ejercicios. Marcombo. <https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/281179>
- Corbí Bellot, A. (2001). Fundamentos de programación. eBook Collection (EBSCOhost).



Introducción a la Programación Avanzada

La programación avanzada se construye sobre los fundamentos básicos, incorporando conceptos, técnicas y herramientas que permiten desarrollar código más eficiente, reutilizable y orientado a problemas reales complejos. Esta presentación refuerza conceptos clave de la programación orientada a objetos, el uso adecuado de entornos de desarrollo integrados (IDE), la configuración de proyectos en lenguajes como C# o Visual Basic, y las prácticas iniciales de documentación del código.

Dominar estos aspectos es esencial para escribir aplicaciones modulares, mantenibles y alineadas con estándares profesionales en el campo de la programación.



Definición y Alcance

La programación avanzada representa un nivel superior en el desarrollo de software donde se aplican técnicas sofisticadas para resolver problemas complejos. No se trata simplemente de escribir código que funcione, sino de crear soluciones elegantes, eficientes y mantenibles que puedan evolucionar con el tiempo.

Este enfoque avanzado requiere un dominio profundo de los paradigmas de programación, especialmente la programación orientada a objetos, así como conocimientos sólidos sobre estructuras de datos, algoritmos eficientes y patrones de diseño.

La asignatura de Programación Avanzada busca proporcionar estas habilidades y conocimientos, preparando a los estudiantes para enfrentar desafíos reales en el desarrollo de software profesional.



El dominio de la programación avanzada permite a los desarrolladores crear aplicaciones robustas que pueden adaptarse a requisitos cambiantes y escalar según las necesidades del negocio, manteniendo siempre la calidad y el rendimiento óptimos.

Conceptos Clave de la Programación Avanzada

1

Programación Orientada a Objetos (POO)

La POO organiza el software en torno a clases que encapsulan atributos y métodos, permitiendo reutilización y extensión mediante mecanismos como la herencia y el polimorfismo. Este paradigma facilita la creación de código modular y mantenible.

2

Entornos de Desarrollo Integrados (IDE)

Un IDE como Visual Studio proporciona un entorno completo para escribir, depurar y administrar proyectos de software, facilitando tareas como la gestión de dependencias, el control de versiones y la ejecución de pruebas automatizadas.

3

Configuración de Proyectos

Implica iniciar la creación de un proyecto, definir el lenguaje de programación y establecer la estructura de carpetas, archivos y bibliotecas necesarias para el desarrollo eficiente de aplicaciones complejas.

4

Documentación de Código

La documentación mejora la mantenibilidad, facilita el trabajo en equipo y asegura que las decisiones de diseño sean comprensibles para otros desarrolladores y para el futuro mantenimiento del software.

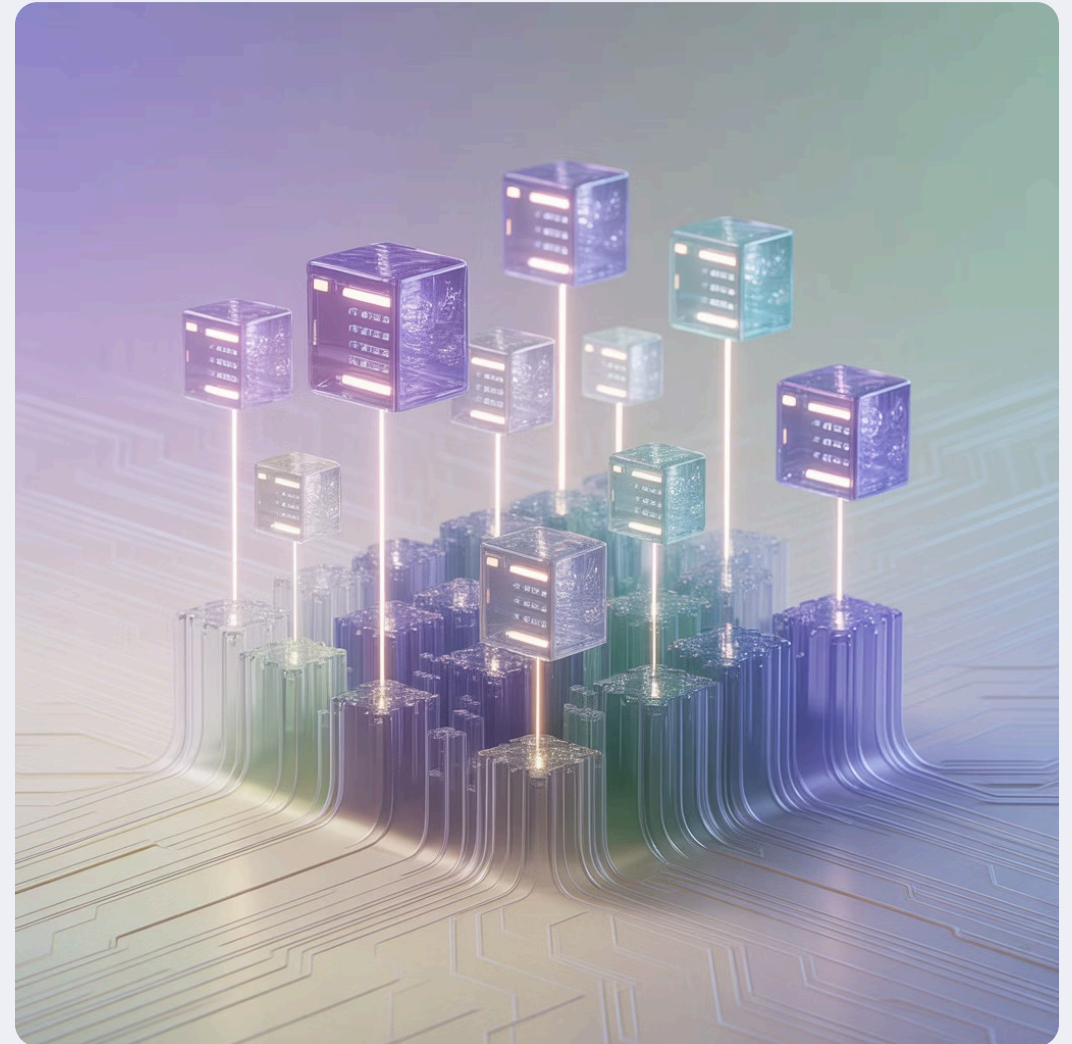
Estos conceptos fundamentales constituyen la base sobre la cual se construyen habilidades más avanzadas en el desarrollo de software profesional.

Programación Orientada a Objetos en Profundidad

Principios Fundamentales de POO

- Encapsulación: Oculta los detalles internos de implementación
- Herencia: Permite la reutilización de código entre clases relacionadas
- Polimorfismo: Facilita que objetos de diferentes clases respondan al mismo mensaje
- Abstracción: Simplifica sistemas complejos enfocándose en lo esencial

La POO permite modelar sistemas complejos de manera más natural, representando entidades del mundo real como objetos con propiedades y comportamientos específicos. Este enfoque facilita el desarrollo de aplicaciones escalables y mantenibles.



La programación orientada a objetos es fundamental en el desarrollo de software moderno, ya que proporciona una estructura clara y organizada para crear aplicaciones complejas. Los lenguajes como C# y Java han sido diseñados específicamente para aprovechar al máximo este paradigma.

Ejemplo Práctico: Clase y Herencia en C#

A continuación se muestra un ejemplo de implementación de clases y herencia en C#, donde se define una clase base **Animal** y una subclase **Perro** que hereda atributos y métodos:

```
public class Animal
{
    public string Nombre { get; set; }

    public void Comer()
    {
        Console.WriteLine($"{Nombre} está comiendo.");
    }
}

public class Perro : Animal
{
    public void Ladrar()
    {
        Console.WriteLine($"{Nombre} está ladrando.");
    }
}

// Uso:
Perro miPerro = new Perro { Nombre = "Fido" };
miPerro.Comer();
miPerro.Ladrar();
```

Análisis del Ejemplo

En este código podemos observar:

- La clase base **Animal** define propiedades y comportamientos comunes
- La clase **Perro** hereda de **Animal** usando el operador ":"
- La herencia permite que **Perro** acceda a la propiedad **Nombre** y al método **Comer()**
- La clase derivada añade comportamiento específico con el método **Ladrar()**

Este ejemplo ilustra cómo la herencia facilita la reutilización de código y la extensión de funcionalidades, principios fundamentales de la programación orientada a objetos.

Documentación de Código en C#

Ejemplo de Documentación XML

```
///
```

▼ `///` Calcula el área de un rectángulo. `///`

```
/// Ancho del rectángulo. /// Alto del rectángulo. /// Área  
calculada. public double CalcularArea(double ancho, double  
alto) { return ancho * alto; }
```

Beneficios de la Documentación

- Mejora la comprensión del código para otros desarrolladores
- Facilita el mantenimiento a largo plazo
- Permite generar documentación automática
- Proporciona información contextual en el IDE
- Establece contratos claros para las interfaces

La documentación adecuada es una práctica esencial en la programación profesional. En C#, los comentarios XML se integran con herramientas como IntelliSense para proporcionar ayuda contextual durante el desarrollo. Además, pueden utilizarse para generar documentación técnica completa utilizando herramientas como Sandcastle o DocFX.

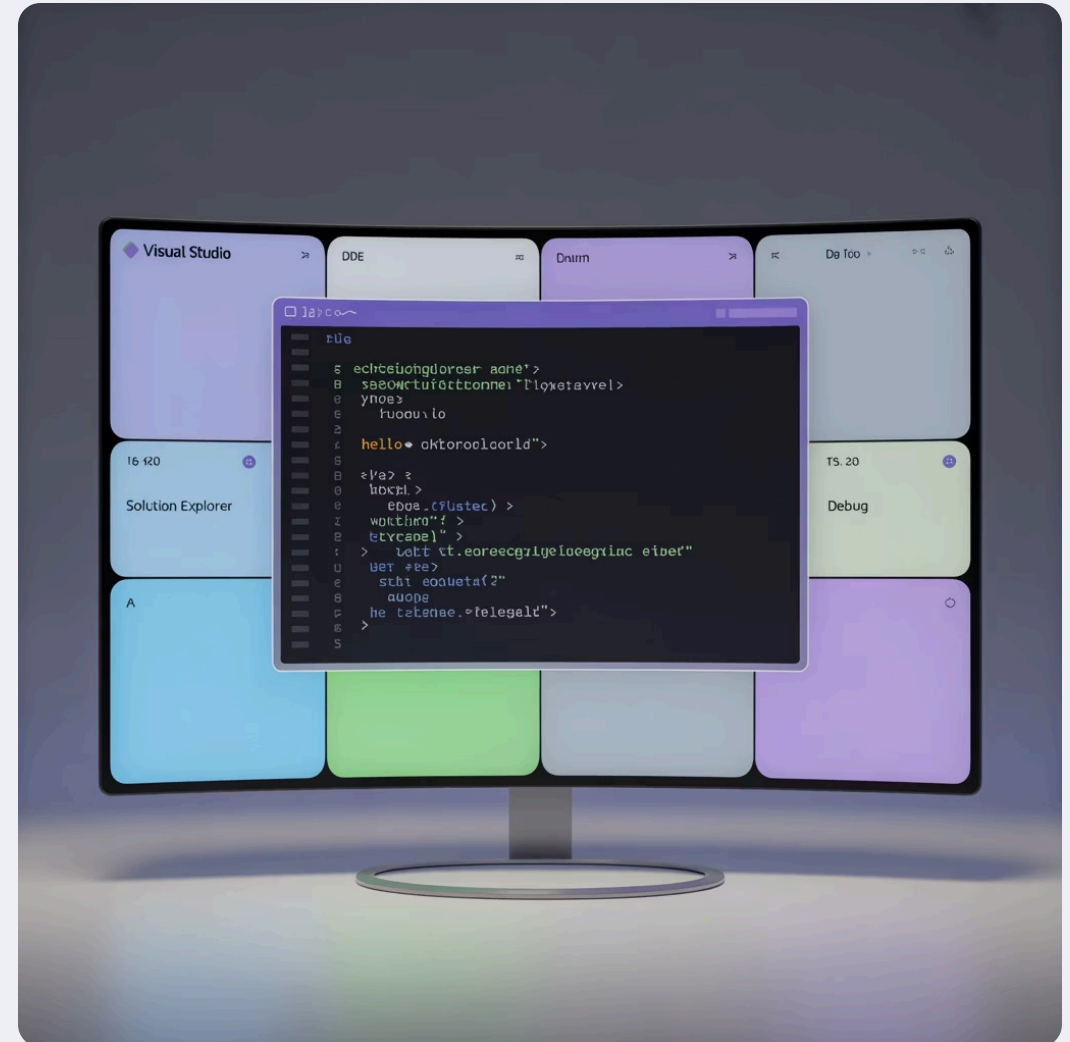
Una buena documentación describe no solo qué hace el código, sino también por qué se implementó de esa manera, facilitando futuras modificaciones y mejoras.

Entornos de Desarrollo Integrados (IDE)

Características Principales de los IDE

- Editor de código con resaltado de sintaxis y autocompletado
- Depurador integrado para identificar y corregir errores
- Herramientas de refactorización para mejorar el código
- Integración con sistemas de control de versiones
- Gestión de dependencias y paquetes
- Compilación y ejecución de proyectos
- Análisis estático de código para detectar problemas potenciales

Los IDE modernos como Visual Studio ofrecen un ecosistema completo que aumenta significativamente la productividad del desarrollador, facilitando tareas que serían complejas o tediosas en un entorno básico.



La elección del IDE adecuado puede tener un impacto significativo en la eficiencia del desarrollo. Visual Studio es uno de los entornos más completos para el desarrollo en C# y .NET, ofreciendo herramientas avanzadas para todas las fases del ciclo de vida del software.

Otros IDE populares incluyen Visual Studio Code (más ligero y versátil), JetBrains Rider (con potentes capacidades de refactorización) y MonoDevelop (para desarrollo multiplataforma).

Conclusiones y Referencias

Conclusiones

La programación avanzada refuerza los fundamentos sobre los cuales se desarrolla el software profesional, destacando la importancia de escribir código modular y legible, configurar correctamente los proyectos y utilizar las herramientas profesionales disponibles.

Estos conocimientos preparan a los estudiantes para abordar retos más complejos en programación, garantizando calidad y profesionalismo en sus soluciones. El dominio de conceptos como la programación orientada a objetos, el uso eficiente de IDE y la documentación adecuada son pilares fundamentales para el éxito en el desarrollo de software.

Referencias (APA 7)

- MARK J. PRICE. (2021). C# 10 and .NET 6 Modern Cross-Platform Development: Build Apps, Websites, and Services with ASP.NET Core 6, Blazor, and EF Core 6 Using Visual Studio 2022 and Visual Studio Code. Packt Publishing.
- Vegas Gertrudix, J. M. (2022). Java 17: programación avanzada. RA-MA.
- Beiroa Mosquera, R. (2018). Aprender Arduino, prototipado y programación avanzada con 100 ejercicios. Marcombo.
- Corbí Bellot, A. (2001). Fundamentos de programación. eBook Collection (EBSCOhost).

INFORMACIÓN GENERAL

Fecha de creación	17-07-2025	Dirección:	DIRECCIÓN DE OPERACIONES
-------------------	------------	------------	--------------------------

Escuela	INGENIERIAS
Programa	INGENIERIA DE SISTEMAS

IDENTIFICACIÓN DEL CURSO

Malla:	VG002
Código:	ISP01
Asignatura:	PROGRAMACION AVANZADA
Semestre (Nivel)	4
Temática:	3

Tema: Interfaz Gráfica de Usuario (GUI) Básica

1. Definición

La interfaz gráfica de usuario (GUI) es el componente visual de una aplicación que permite a los usuarios interactuar con el sistema de manera intuitiva a través de elementos como botones, cuadros de texto, etiquetas, menús y formularios. En la programación avanzada, diseñar una GUI implica no solo conocer los controles disponibles, sino también manejar eventos, validar entradas y asegurar una experiencia de usuario fluida.

Esta unidad introduce la creación de formularios básicos, el uso de controles fundamentales y la gestión de eventos para capturar las acciones del usuario, empleando entornos como Visual Studio y lenguajes como C#.

2. Conceptos clave

- Creación y manejo de formularios simples: Un formulario es una ventana que contiene los controles que permiten al usuario interactuar con la aplicación. Su creación incluye definir el tamaño, el título y los controles iniciales.

- **Uso básico de controles GUI:** Los controles son componentes visuales predefinidos, como botones (Button), etiquetas (Label), cuadros de texto (TextBox) y casillas de verificación (CheckBox), que permiten presentar información y recibir entradas del usuario.
 - **Captura de eventos básicos:** Un evento es una acción que el usuario realiza sobre la GUI, como hacer clic en un botón o escribir en un cuadro de texto. La programación de eventos consiste en asociar funciones (manejadores) a estos eventos.
 - **Validación sencilla de entradas:** Verifica que los datos introducidos por el usuario cumplan con los formatos esperados, evitando errores o comportamientos inesperados.
-

3. Ejemplos en texto

- **Ejemplo 1: Formulario básico en Windows Forms.** Crear un formulario simple con un botón que muestra un mensaje cuando es pulsado:

```
public partial class Form1 : Form
{
    public Form1()
    {
        InitializeComponent();
    }

    private void btnSaludar_Click(object sender, EventArgs e)
    {
        MessageBox.Show("¡Hola, bienvenido a la aplicación!");
    }
}
```

En el diseñador de Visual Studio:

- Arrastra un botón (Button) al formulario.
 - Asigna el nombre btnSaludar.
 - Doble clic sobre el botón para generar el evento Click y escribir el código anterior.
- **Ejemplo 2: Validación de entrada en un cuadro de texto.** Verifica que el usuario no deje vacío un TextBox antes de procesar:

```
private void btnProcesar_Click(object sender, EventArgs e)
{

```

```
if(string.IsNullOrEmpty(txtNombre.Text))  
  
{  
  
    MessageBox.Show("Por favor, ingrese su nombre.");  
  
    return;  
  
}  
  
MessageBox.Show($"Nombre ingresado: {txtNombre.Text}");  
  
}
```

- Ejemplo 3: Captura de evento doble clic. Puedes manejar un evento de doble clic en un ListBox para mostrar el elemento seleccionado:

```
private void listBox1_DoubleClick(object sender, EventArgs e)  
  
{  
  
    if(listBox1.SelectedItem != null)  
  
    {  
  
        MessageBox.Show($"Seleccionaste: {listBox1.SelectedItem.ToString()}");  
  
    }  
  
}
```

4. Conclusión

Dominar la creación de interfaces gráficas básicas permite a los estudiantes desarrollar aplicaciones interactivas que son más accesibles y amigables para el usuario final. Comprender cómo manejar controles, eventos y validar datos es el primer paso para diseñar sistemas robustos y con buena experiencia de usuario. Estas habilidades son esenciales en la construcción de aplicaciones de escritorio y web modernas.

5. Referencias (APA 7)

- MARK J. PRICE. (2021). C# 10 and .NET 6 – Modern Cross-Platform Development: Build Apps, Websites, and Services with ASP.NET Core 6, Blazor, and EF Core 6 Using Visual Studio 2022 and Visual Studio Code. Packt Publishing.
- Vegas Gertrudix, J. M. (2022). Java 17: programación avanzada. RA-MA.
- Beiroa Mosquera, R. (2018). Aprender Arduino, prototipado y programación avanzada con 100 ejercicios. Marcombo. <https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/281179>
- Corbí Bellot, A. (2001). Fundamentos de programación. eBook Collection (EBSCOhost).





Unlock insights, drive growth

Visualize and analyze your data with ease

Trem air flow/c

Interfaz Gráfica de Usuario (GUI)

Básica

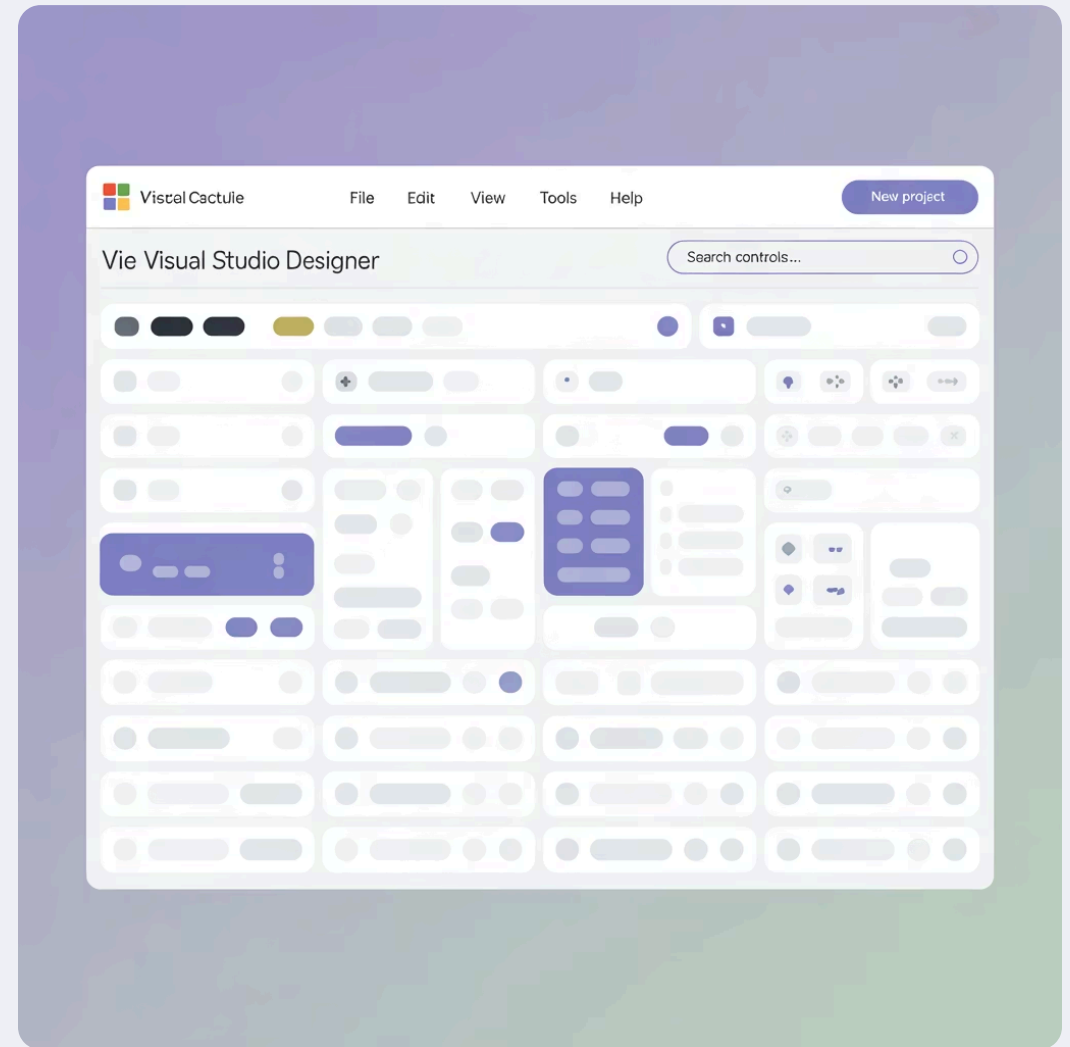
Bienvenidos a esta presentación sobre Interfaces Gráficas de Usuario (GUI) en Programación Avanzada. Exploraremos los fundamentos para crear aplicaciones interactivas y amigables, desde la creación de formularios hasta la gestión de eventos y validación de datos. Estos conocimientos son esenciales para desarrollar sistemas robustos con una experiencia de usuario fluida.

¿Qué es una Interfaz Gráfica de Usuario?

La interfaz gráfica de usuario (GUI) es el componente visual de una aplicación que permite a los usuarios interactuar con el sistema de manera intuitiva a través de elementos como botones, cuadros de texto, etiquetas, menús y formularios.

En la programación avanzada, diseñar una GUI implica no solo conocer los controles disponibles, sino también manejar eventos, validar entradas y asegurar una experiencia de usuario fluida.

Esta unidad introduce la creación de formularios básicos, el uso de controles fundamentales y la gestión de eventos para capturar las acciones del usuario, empleando entornos como Visual Studio y lenguajes como C#.



El diseño de interfaces gráficas es fundamental para crear aplicaciones que sean accesibles y fáciles de usar para el usuario final, mejorando significativamente la experiencia de interacción con el software.

Conceptos Clave en el Desarrollo de GUI

1

Creación y Manejo de Formularios

Un formulario es una ventana que contiene los controles que permiten al usuario interactuar con la aplicación. Su creación incluye definir el tamaño, el título y los controles iniciales que contendrá la ventana principal de la aplicación.

2

Uso Básico de Controles GUI

Los controles son componentes visuales predefinidos, como botones (Button), etiquetas (Label), cuadros de texto (TextBox) y casillas de verificación (CheckBox), que permiten presentar información y recibir entradas del usuario de manera estructurada.

3

Captura de Eventos Básicos

Un evento es una acción que el usuario realiza sobre la GUI, como hacer clic en un botón o escribir en un cuadro de texto. La programación de eventos consiste en asociar funciones (manejadores) a estos eventos para responder a las interacciones.

4

Validación Sencilla de Entradas

Verifica que los datos introducidos por el usuario cumplan con los formatos esperados, evitando errores o comportamientos inesperados en la ejecución del programa.

Ejemplo 1: Formulario Básico en Windows Forms

Crear un formulario simple con un botón que muestra un mensaje cuando es pulsado:

```
public partial class Form1 : Form
{
    public Form1()
    {
        InitializeComponent();
    }

    private void btnSaludar_Click(object sender, EventArgs e)
    {
        MessageBox.Show("¡Hola, bienvenido a la aplicación!");
    }
}
```

En el diseñador de Visual Studio:

- Arrastra un botón (Button) al formulario
- Asigna el nombre btnSaludar
- Doble clic sobre el botón para generar el evento Click y escribir el código anterior



Este ejemplo muestra la estructura básica de un formulario Windows Forms con un botón que responde al evento Click mostrando un mensaje de bienvenida al usuario.

Ejemplo 2: Validación de Entrada en un Cuadro de Texto

Verifica que el usuario no deje vacío un TextBox antes de procesar:

```
private void btnProcesar_Click(object sender, EventArgs e)
{
    if(string.IsNullOrEmpty(txtNombre.Text))
    {
        MessageBox.Show("Por favor, ingrese su nombre.");
        return;
    }
    MessageBox.Show($"Nombre ingresado: {txtNombre.Text}");
}
```

Este código implementa una validación básica que:

- Comprueba si el cuadro de texto está vacío
- Muestra un mensaje de error si no se ha introducido texto
- Procesa el texto solo cuando hay contenido válido

Signup Foonup

you llgmd yorurp

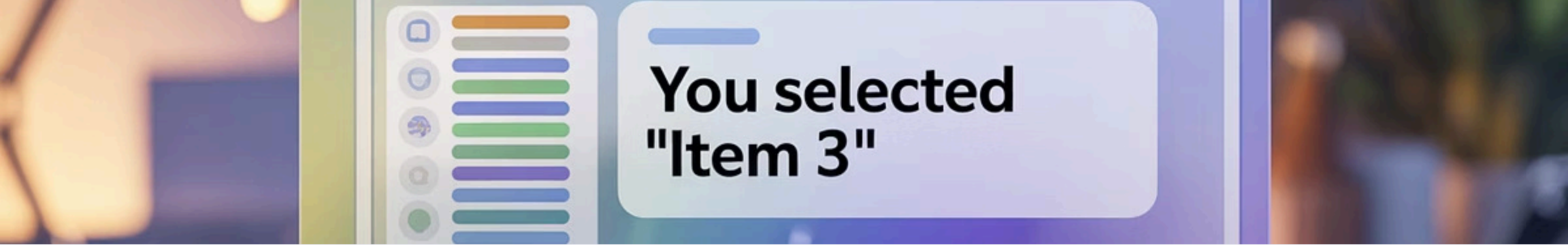
Email address

Create Account

Looks good! ✓

[Terms of Service](#) [Privacy Policy](#) × [Copyright atone teonce](#) ACME CORP

La validación de entradas es crucial para prevenir errores en la aplicación y mejorar la experiencia del usuario proporcionando retroalimentación inmediata sobre problemas en los datos introducidos.



You selected
"Item 3"

Ejemplo 3: Captura de Evento Doble Clic

Código del Evento

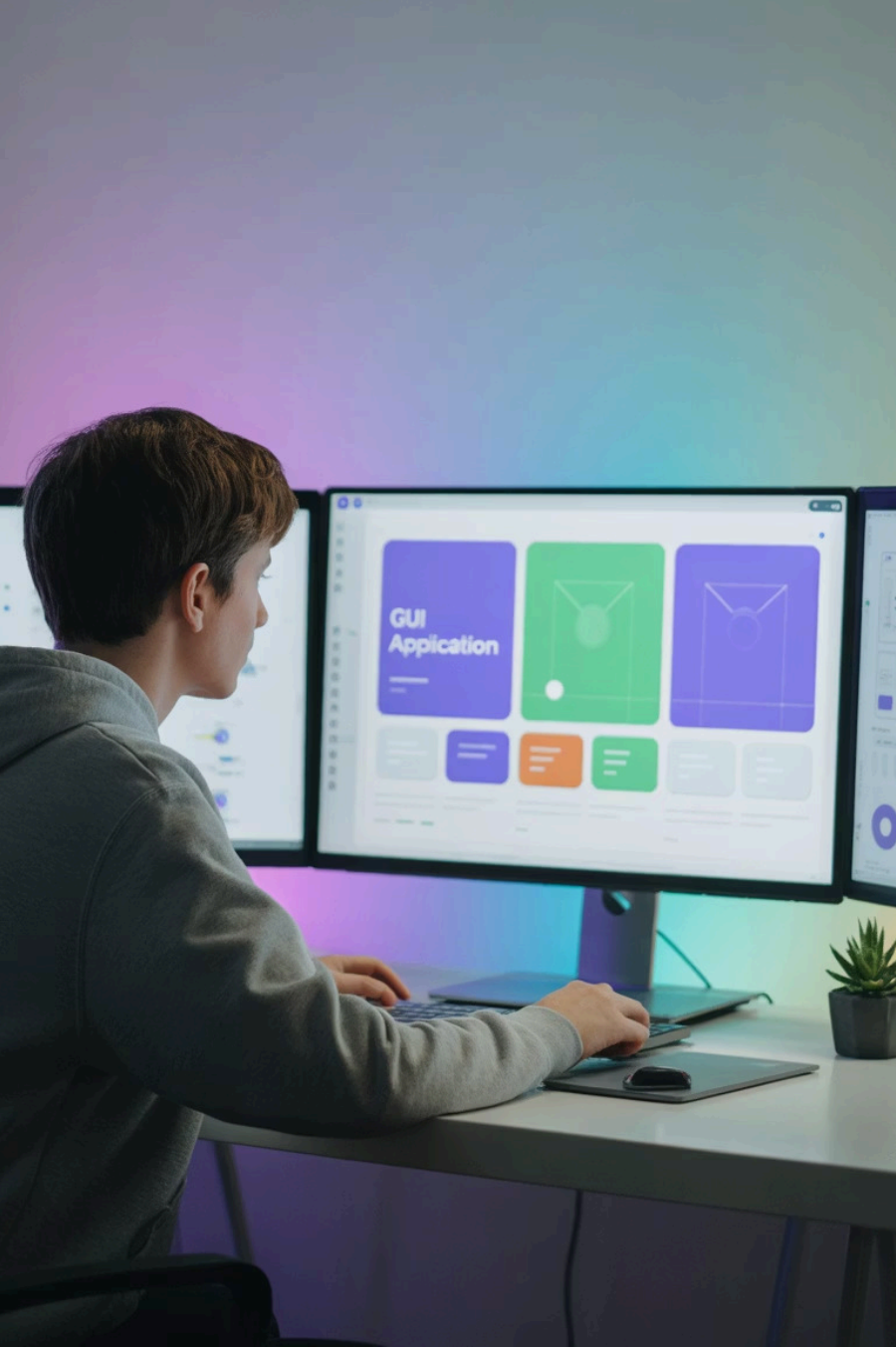
```
private void listBox1_DoubleClick(object sender, EventArgs e)
{
    if(listBox1.SelectedItem != null)
    {
        MessageBox.Show($"Seleccionaste:
{listBox1.SelectedItem.ToString()}");
    }
}
```

Explicación

Este ejemplo muestra cómo manejar un evento de doble clic en un ListBox para mostrar el elemento seleccionado:

- Se verifica que haya un elemento seleccionado
- Se muestra un mensaje con el texto del elemento
- El evento se dispara solo cuando el usuario hace doble clic

Los eventos de doble clic son útiles para implementar acciones secundarias o confirmaciones, permitiendo una interacción más rica con los elementos de la interfaz.



Conclusiones

Dominar la creación de interfaces gráficas básicas permite a los estudiantes desarrollar aplicaciones interactivas que son más accesibles y amigables para el usuario final. Comprender cómo manejar controles, eventos y validar datos es el primer paso para diseñar sistemas robustos y con buena experiencia de usuario.

Estas habilidades son esenciales en la construcción de aplicaciones de escritorio y web modernas, y constituyen la base para avanzar hacia técnicas más complejas de desarrollo de interfaces.

Beneficios

- Aplicaciones más intuitivas
- Mejor experiencia de usuario
- Reducción de errores
- Mayor productividad

Próximos Pasos

- Interfaces responsivas
- Controles avanzados
- Patrones de diseño GUI
- Accesibilidad



Referencias (APA 7)

MARK J. PRICE. (2021). C# 10 and .NET 6 Modern Cross-Platform Development: Build Apps, Websites, and Services with ASP.NET Core 6, Blazor, and EF Core 6 Using Visual Studio 2022 and Visual Studio Code. Packt Publishing.

Vegas Gertrudix, J. M. (2022). Java 17: programación avanzada. RA-MA.

Beiroa Mosquera, R. (2018). Aprender Arduino, prototipado y programación avanzada con 100 ejercicios. Marcombo.

<https://elibronet.zproxy.cun.edu.co/es/lc/bibliocun/titulos/281179>

Corbí Bellot, A. (2001). Fundamentos de programación. eBook Collection (EBSCOhost).

Estas referencias proporcionan información adicional y recursos para profundizar en el desarrollo de interfaces gráficas y programación avanzada.

INFORMACIÓN GENERAL

Fecha de creación	17-07-2025	Dirección:	DIRECCIÓN DE OPERACIONES
-------------------	------------	------------	--------------------------

Escuela	INGENIERIAS
Programa	INGENIERIA DE SISTEMAS

IDENTIFICACIÓN DEL CURSO

Malla:	VG002
Código:	ISP01
Asignatura:	PROGRAMACION AVANZADA
Semestre (Nivel)	4
Temática:	4

Tema: Manejo de Colecciones y Arreglos

1. Definición

El manejo de colecciones y arreglos consiste en la utilización de estructuras de datos que permiten almacenar y manipular conjuntos de elementos, ya sean homogéneos o heterogéneos. En programación avanzada, es esencial entender cómo funcionan los arreglos unidimensionales, listas dinámicas y otras colecciones para recorrer datos, realizar búsquedas y procesar información de manera eficiente.

Las colecciones y arreglos son fundamentales para organizar datos temporales o persistentes en aplicaciones como inventarios, listas de estudiantes, catálogos de productos y más.

2. Conceptos clave

- Arreglos unidimensionales: Estructura básica para almacenar elementos del mismo tipo, con tamaño fijo y acceso a cada elemento por su índice.
- Listas genéricas: Colecciones dinámicas que permiten agregar o eliminar elementos sin necesidad de definir un tamaño fijo al momento de su declaración, facilitando la manipulación de datos.

- Recorrido y búsqueda secuencial: Procesos para iterar sobre todos los elementos de una colección o arreglo y localizar un valor específico, generalmente usando bucles.
 - Aplicaciones prácticas: Escenarios reales donde se usan estas estructuras para resolver problemas cotidianos, como gestionar inventarios, mantener registros de estudiantes o controlar listas de tareas.
-

3. Ejemplos en texto

- Ejemplo 1: Arreglo unidimensional para almacenar calificaciones. Un arreglo fijo de 5 posiciones almacena las calificaciones de un estudiante:

```
int[] calificaciones = { 90, 85, 78, 92, 88 };

for(int i = 0; i < calificaciones.Length; i++)

{

    Console.WriteLine($"Calificación {i+1}: {calificaciones[i]}");

}
```

- Ejemplo 2: Búsqueda secuencial en un arreglo. Busca un número en un arreglo de enteros:

```
int[] numeros = { 3, 7, 10, 25, 42 };

int buscado = 25;

bool encontrado = false;

foreach(int n in numeros)

{

    if(n == buscado)

    {

        encontrado = true;

        break;

    }

}
```

- Ejemplo 3: Lista genérica para manejar inventario. Una lista dinámica para agregar productos y mostrar el inventario:

```
List<string> inventario = new List<string>();

inventario.Add("Teclado");

inventario.Add("Monitor");
```

```
inventario.Add("Mouse");

foreach(string producto in inventario)

{

    Console.WriteLine($"Producto: {producto}");

}

inventario.Remove("Mouse");

Console.WriteLine("Después de eliminar Mouse:");

foreach(string producto in inventario)

{

    Console.WriteLine($"Producto: {producto}");

}
```

4. Conclusión

El manejo eficiente de colecciones y arreglos permite al programador almacenar, recorrer y procesar datos de manera estructurada, facilitando la implementación de soluciones para problemas cotidianos. Esta unidad proporciona las bases para trabajar con estructuras de datos dinámicas, optimizar el rendimiento y preparar al estudiante para estructuras más complejas en etapas posteriores.

5. Referencias (APA 7)

- MARK J. PRICE. (2021). C# 10 and .NET 6 – Modern Cross-Platform Development: Build Apps, Websites, and Services with ASP.NET Core 6, Blazor, and EF Core 6 Using Visual Studio 2022 and Visual Studio Code. Packt Publishing.
- Vegas Gertrudix, J. M. (2022). Java 17: programación avanzada. RA-MA.
- Beiroa Mosquera, R. (2018). Aprender Arduino, prototipado y programación avanzada con 100 ejercicios. Marcombo. <https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/281179>
- Márquez, F. (2015). Unix: programación avanzada. RA-MA.

Manejo de Colecciones y Arreglos en Programación Avanzada

Bienvenidos a esta presentación sobre el manejo de colecciones y arreglos, un componente fundamental en la programación avanzada. Exploraremos cómo estas estructuras de datos nos permiten almacenar, manipular y procesar información de manera eficiente, desde los arreglos unidimensionales básicos hasta las listas dinámicas más flexibles. A lo largo de esta presentación, veremos definiciones clave, conceptos fundamentales, ejemplos prácticos y aplicaciones en escenarios reales.

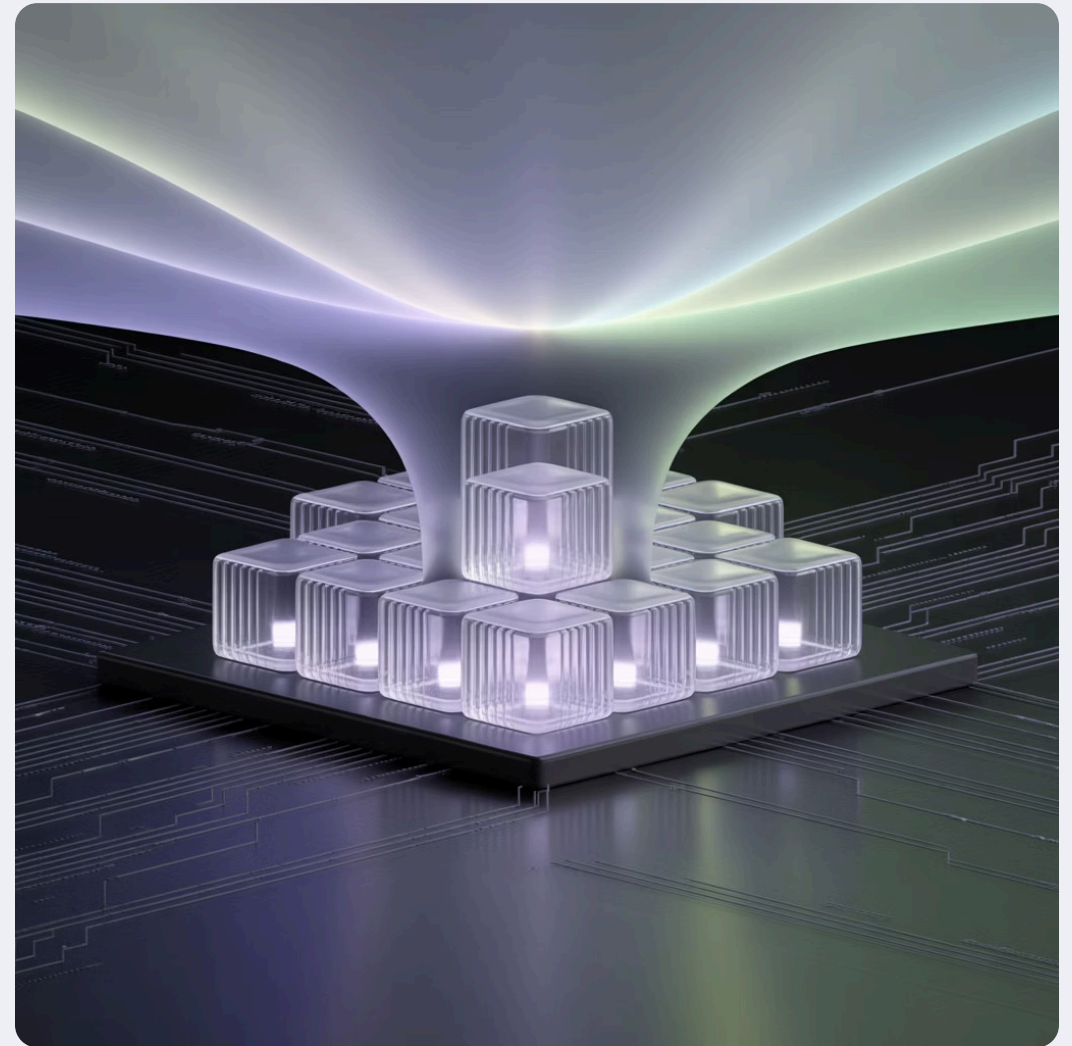


Definición y Fundamentos

El manejo de colecciones y arreglos consiste en la utilización de estructuras de datos que permiten almacenar y manipular conjuntos de elementos, ya sean homogéneos o heterogéneos. En programación avanzada, es esencial entender cómo funcionan estas estructuras para recorrer datos, realizar búsquedas y procesar información de manera eficiente.

Estas estructuras son fundamentales para organizar datos temporales o persistentes en aplicaciones como:

- Inventarios de productos
- Listas de estudiantes
- Catálogos de productos
- Registros de transacciones
- Sistemas de gestión de tareas



La elección adecuada de la estructura de datos puede tener un impacto significativo en el rendimiento y la eficiencia de nuestras aplicaciones, especialmente cuando trabajamos con grandes volúmenes de información.

Conceptos Clave

1

Arreglos Unidimensionales

Estructura básica para almacenar elementos del mismo tipo, con tamaño fijo y acceso a cada elemento por su índice. Proporcionan acceso directo a los elementos mediante operaciones $O(1)$.

2

Listas Genéricas

Colecciones dinámicas que permiten agregar o eliminar elementos sin necesidad de definir un tamaño fijo al momento de su declaración, facilitando la manipulación de datos y adaptándose a necesidades cambiantes.

3

Recorrido y Búsqueda Secuencial

Procesos para iterar sobre todos los elementos de una colección o arreglo y localizar un valor específico, generalmente usando bucles como for, foreach o while para examinar cada elemento.

4

Aplicaciones Prácticas

Escenarios reales donde se usan estas estructuras para resolver problemas cotidianos, como gestionar inventarios, mantener registros de estudiantes o controlar listas de tareas en aplicaciones empresariales.

Arreglos Unidimensionales

Características Principales

- Tamaño fijo definido en la declaración
- Elementos del mismo tipo de datos
- Acceso directo mediante índices (0-based)
- Almacenamiento contiguo en memoria

Los arreglos son ideales cuando conocemos de antemano la cantidad exacta de elementos que necesitamos almacenar y cuando requerimos un acceso rápido a posiciones específicas.

Ejemplo: Arreglo de Calificaciones

```
int[] calificaciones = { 90, 85, 78, 92, 88
};

for(int i = 0; i < calificaciones.Length;
i++)
{
    Console.WriteLine($"Calificación
{i+1}:
    {calificaciones[i]}");
}
```

Este ejemplo muestra cómo declarar un arreglo fijo de 5 posiciones para almacenar las calificaciones de un estudiante y luego recorrerlo para mostrar cada valor utilizando un bucle for tradicional.

Búsqueda Secuencial en Arreglos

Proceso de Búsqueda

La búsqueda secuencial es una técnica fundamental que consiste en examinar cada elemento de un arreglo o colección hasta encontrar el valor buscado o determinar que no existe. Es simple de implementar pero puede ser ineficiente para grandes conjuntos de datos.

- Complejidad: $O(n)$ en el peor caso
- Funciona en arreglos ordenados y desordenados
- No requiere estructura adicional

Ejemplo: Búsqueda de un Número

```
int[] numeros = { 3, 7, 10, 25, 42 };
int buscado = 25;
bool encontrado = false;

foreach(int n in numeros)
{
    if(n == buscado)
    {
        encontrado = true;
        break;
    }
}

Console.WriteLine(encontrado ?
    "Número encontrado" : "Número no encontrado");
```



Inicio de la búsqueda

Se define el valor a buscar y se prepara para recorrer el arreglo



Iteración

Se examina cada elemento del arreglo secuencialmente



Comparación

Se compara cada elemento con el valor buscado



Resultado

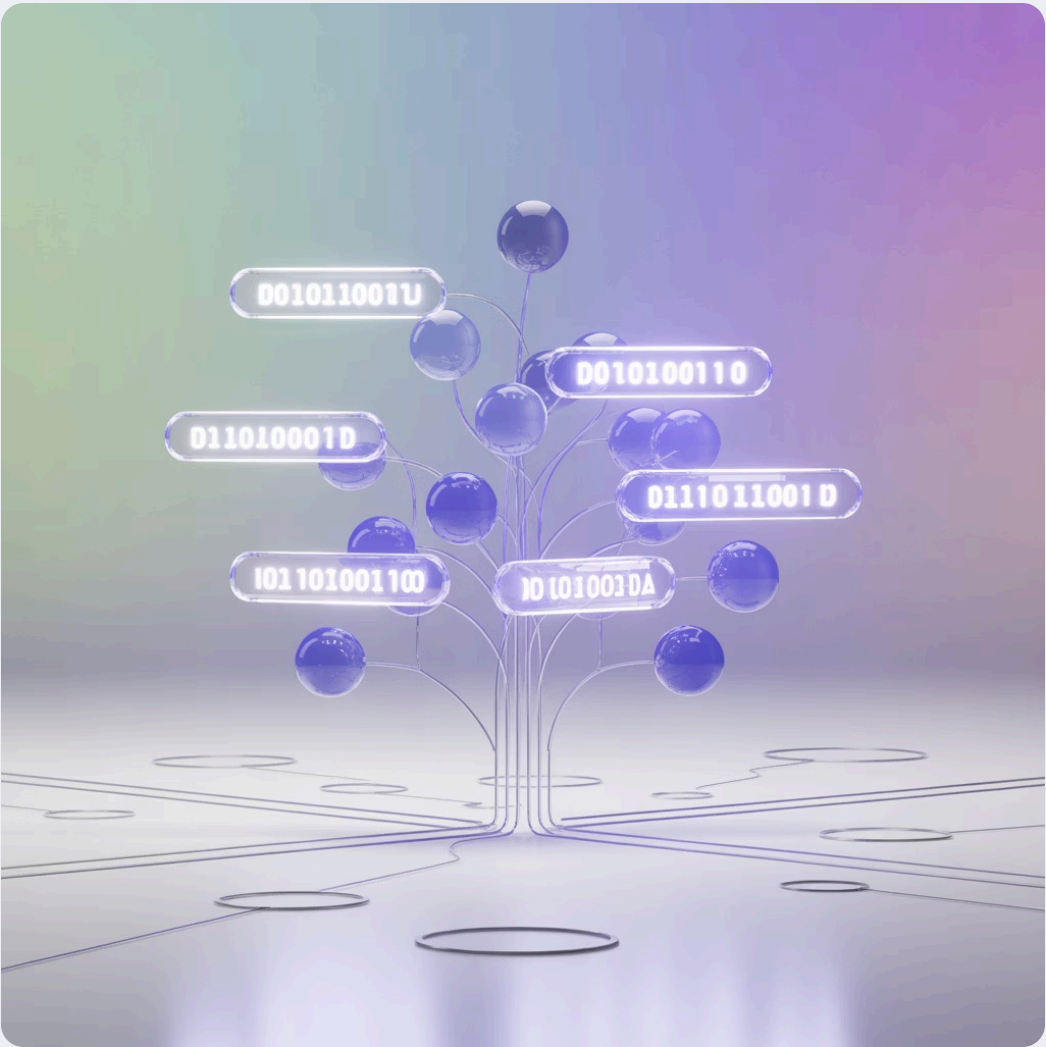
Se determina si el valor fue encontrado o no

Listas Genéricas

Ventajas de las Listas

- Tamaño dinámico que crece según necesidad
- Métodos integrados para agregar, eliminar y buscar
- Tipado genérico para mayor seguridad
- Mayor flexibilidad que los arreglos tradicionales

Las listas genéricas son especialmente útiles cuando no conocemos de antemano el número exacto de elementos que necesitaremos almacenar o cuando necesitamos frecuentemente agregar o eliminar elementos.



Ejemplo: Lista para Gestión de Inventario

```
List inventario = new List();

// Agregar productos
inventario.Add("Teclado");
inventario.Add("Monitor");
inventario.Add("Mouse");

// Mostrar inventario
foreach(string producto in inventario) {
    Console.WriteLine($"Producto: {producto}");
}

// Eliminar un producto
inventario.Remove("Mouse");

// Mostrar inventario actualizado
Console.WriteLine("Después de eliminar Mouse:");
foreach(string producto in inventario) {
    Console.WriteLine($"Producto: {producto}");
}
```

Aplicaciones Prácticas

Gestión de Inventarios

Utilización de listas para mantener un registro dinámico de productos, permitiendo agregar nuevos artículos, eliminar los agotados y actualizar existencias en tiempo real.

Sistemas Educativos

Arreglos y listas para almacenar información de estudiantes, calificaciones y asistencias, facilitando el cálculo de promedios y la generación de reportes académicos.

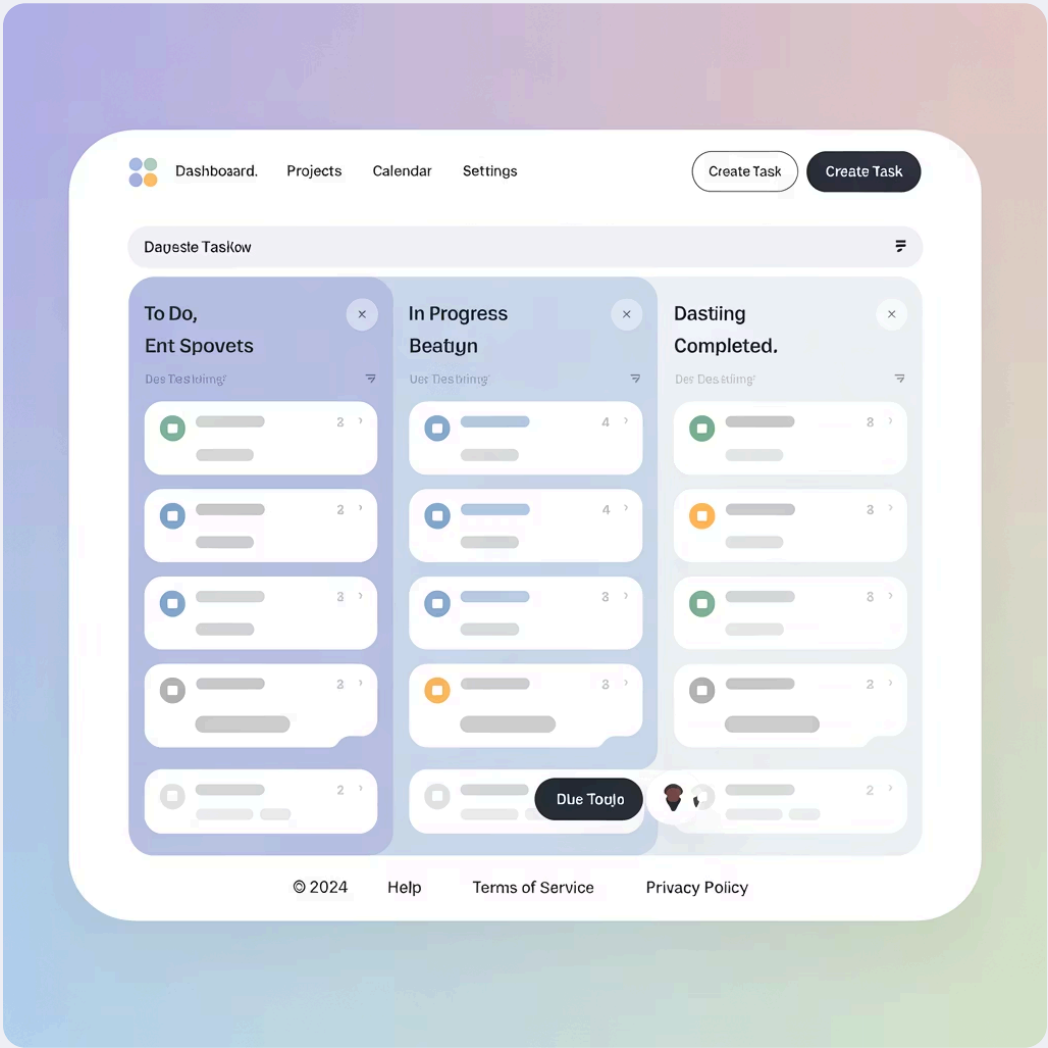
Aplicaciones Financieras

Estructuras para registrar transacciones, calcular balances y analizar patrones de gasto utilizando recorridos y búsquedas eficientes.

Caso de Estudio: Sistema de Gestión de Tareas

Un sistema de gestión de tareas puede implementarse utilizando listas genéricas para almacenar las tareas pendientes, completadas y en progreso. Cada tarea puede ser un objeto con propiedades como título, descripción, fecha límite y estado.

La flexibilidad de las listas permite agregar nuevas tareas, marcarlas como completadas (moviéndolas a otra lista) y ordenarlas según diferentes criterios como prioridad o fecha.



Las colecciones y arreglos son fundamentales en prácticamente todas las aplicaciones modernas, desde simples herramientas de productividad hasta complejos sistemas empresariales.

Conclusiones y Referencias

Conclusiones

El manejo eficiente de colecciones y arreglos permite al programador almacenar, recorrer y procesar datos de manera estructurada, facilitando la implementación de soluciones para problemas cotidianos. Esta unidad proporciona las bases para trabajar con estructuras de datos dinámicas, optimizar el rendimiento y preparar al estudiante para estructuras más complejas en etapas posteriores.

La elección entre arreglos y colecciones dinámicas dependerá de los requisitos específicos de cada aplicación, considerando factores como el tamaño de los datos, la frecuencia de las operaciones de inserción/eliminación y los patrones de acceso.

Referencias (APA 7)

- MARK J. PRICE. (2021). C# 10 and .NET 6 Modern Cross-Platform Development: Build Apps, Websites, and Services with ASP.NET Core 6, Blazor, and EF Core 6 Using Visual Studio 2022 and Visual Studio Code. Packt Publishing.
- Vegas Gertrudix, J. M. (2022). Java 17: programación avanzada. RA-MA.
- Beiroa Mosquera, R. (2018). Aprender Arduino, prototipado y programación avanzada con 100 ejercicios. Marcombo.
- Márquez, F. (2015). Unix: programación avanzada. RA-MA.

Próximos Pasos

Explorar estructuras de datos más avanzadas como árboles, grafos y tablas hash para resolver problemas más complejos y optimizar el rendimiento de las aplicaciones.

Ejercicios Prácticos

Implementar un sistema de gestión utilizando las estructuras estudiadas para consolidar los conocimientos adquiridos y desarrollar habilidades prácticas.

Recursos Adicionales

Consultar la documentación oficial de los lenguajes de programación para profundizar en las implementaciones específicas de colecciones y arreglos.

INFORMACIÓN GENERAL

Fecha de creación	17-07-2025	Dirección:	DIRECCIÓN DE OPERACIONES
-------------------	------------	------------	--------------------------

Escuela	INGENIERIAS
Programa	INGENIERIA DE SISTEMAS

IDENTIFICACIÓN DEL CURSO

Malla:	VG002
Código:	ISP01
Asignatura:	PROGRAMACION AVANZADA
Semestre (Nivel)	4
Temática:	5

Tema: Bases de Datos Relacionales Básicas y Consultas Avanzadas

1. Definición

Las bases de datos relacionales (BDR) son sistemas que almacenan y gestionan datos organizados en tablas relacionadas entre sí mediante claves primarias y foráneas. En el contexto de la programación avanzada, es fundamental comprender cómo diseñar, conectar y consultar bases de datos desde una aplicación, implementando operaciones CRUD básicas y consultas avanzadas para obtener información eficiente y relevante.

Esta unidad introduce la conexión a una BDR desde un lenguaje de programación, la ejecución de sentencias SQL para gestión de datos, el uso de JOIN para combinar datos de múltiples tablas y buenas prácticas para optimizar consultas en bases pequeñas.

2. Conceptos clave

- Modelo relacional: Organiza los datos en tablas con filas (registros) y columnas (campos), asegurando integridad mediante restricciones como claves primarias y foráneas.

- Conexión a bases de datos desde Visual Studio: Consiste en configurar la cadena de conexión que permite a la aplicación comunicarse con el servidor de base de datos.
 - Sentencias CRUD: Son las operaciones esenciales para Crear (INSERT), Leer (SELECT), Actualizar (UPDATE) y Eliminar (DELETE) registros.
 - Consultas avanzadas con filtros (WHERE): Permiten seleccionar datos que cumplen condiciones específicas, optimizando la precisión de los resultados.
 - Consultas multi-tabla (JOIN): Combina datos de más de una tabla para producir resultados más completos.
 - Optimización inicial: Uso de índices y revisión de consultas para mejorar rendimiento.
-

3. Ejemplos en texto

- Ejemplo 1: Creación de tabla e inserción de datos:

```
CREATE TABLE Estudiantes (  
    Id INT PRIMARY KEY IDENTITY,  
    Nombre NVARCHAR(50),  
    Edad INT,  
    Carrera NVARCHAR(50)  
);  
  
INSERT INTO Estudiantes (Nombre, Edad, Carrera)  
VALUES ('Ana Torres', 21, 'Ingeniería de Sistemas');
```

- Ejemplo 2: Conexión a SQL Server desde C#

```
using System.Data.SqlClient;  
  
string conexion =  
"Server=localhost;Database=Universidad;Trusted_Connection=True;";  
  
using(SqlConnection conn = new SqlConnection(conexion))  
{  
    conn.Open();  
    Console.WriteLine("Conexión exitosa.");  
}
```

- Ejemplo 3: Consulta con filtro:

```
SELECT Nombre, Carrera
```

FROM Estudiantes

WHERE Edad > 20;

- Ejemplo 4: Consulta JOIN en dos tablas

SELECT e.Nombre, c.NombreCurso

FROM Estudiantes e

JOIN Matriculas m ON e.Id = m.Estudianteld

JOIN Cursos c ON m.Cursold = c.Id;

- Ejemplo 5: Índice para optimizar búsqueda

CREATE INDEX idx_Edad

ON Estudiantes (Edad);

4. Conclusión

Las bases de datos relacionales son esenciales para aplicaciones que gestionan grandes volúmenes de datos estructurados. Esta unidad capacita al estudiante para modelar tablas, establecer relaciones, realizar consultas simples y avanzadas, e integrar la base de datos con sus aplicaciones. Dominar estas técnicas garantiza que los datos sean precisos, consistentes y estén disponibles para su uso eficiente

5. Referencias (APA 7)

- MARK J. PRICE. (2021). C# 10 and .NET 6 – Modern Cross-Platform Development: Build Apps, Websites, and Services with ASP.NET Core 6, Blazor, and EF Core 6 Using Visual Studio 2022 and Visual Studio Code. Packt Publishing.
- Márquez, F. (2015). Unix: programación avanzada. RA-MA.
- Rincón Cardona, J. J. (2025). Aprenda desarrollo web con Laravel desde cero. Marcombo. <https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/281767>
- Corbí Bellot, A. (2001). Fundamentos de programación. eBook Collection (EBSCOhost).

Relational Database Architecture



Bases de Datos Relacionales Básicas y Consultas Avanzadas

Asignatura: PROGRAMACIÓN AVANZADA

Definición y Fundamentos

Las bases de datos relacionales (BDR) son sistemas que almacenan y gestionan datos organizados en tablas relacionadas entre sí mediante claves primarias y foráneas.

En el contexto de la programación avanzada, es fundamental comprender cómo diseñar, conectar y consultar bases de datos desde una aplicación, implementando operaciones CRUD básicas y consultas avanzadas para obtener información eficiente y relevante.



Conceptos Clave

1

Modelo Relacional

Organiza los datos en tablas con filas (registros) y columnas (campos), asegurando integridad mediante restricciones como claves primarias y foráneas.

2

Conexión a Bases de Datos

Consiste en configurar la cadena de conexión que permite a la aplicación comunicarse con el servidor de base de datos.

3

Operaciones CRUD

Son las operaciones esenciales para Crear (INSERT), Leer (SELECT), Actualizar (UPDATE) y Eliminar (DELETE) registros.

Consultas y Optimización

Consultas con Filtros (WHERE)

Permiten seleccionar datos que cumplen condiciones específicas, optimizando la precisión de los resultados.

Consultas Multi-tabla (JOIN)

Combina datos de más de una tabla para producir resultados más completos.

Optimización Inicial

Uso de índices y revisión de consultas para mejorar rendimiento.



Ejemplo: Creación de Tabla e Inserción

```
CREATE TABLE Estudiantes (  
  Id INT PRIMARY KEY IDENTITY,  
  Nombre NVARCHAR(50),  
  Edad INT,  
  Carrera NVARCHAR(50)  
);
```

```
INSERT INTO Estudiantes (Nombre, Edad, Carrera)  
VALUES ('Ana Torres', 21, 'Ingeniería de Sistemas');
```

Este ejemplo muestra la creación de una tabla de Estudiantes con campos para Id, Nombre, Edad y Carrera, seguido de la inserción de un registro.

Conexión y Consultas

Conexión a SQL Server desde C#

```
using System.Data.SqlClient;
string conexion =
"Server=localhost;Database=Universidad;Trusted_Connection=True;";
using(SqlConnection conn = new SqlConnection(conexion))
{
    conn.Open();
    Console.WriteLine("Conexión exitosa.");
}
```

Consulta con Filtro

```
SELECT Nombre, Carrera
FROM Estudiantes
WHERE Edad > 20;
```

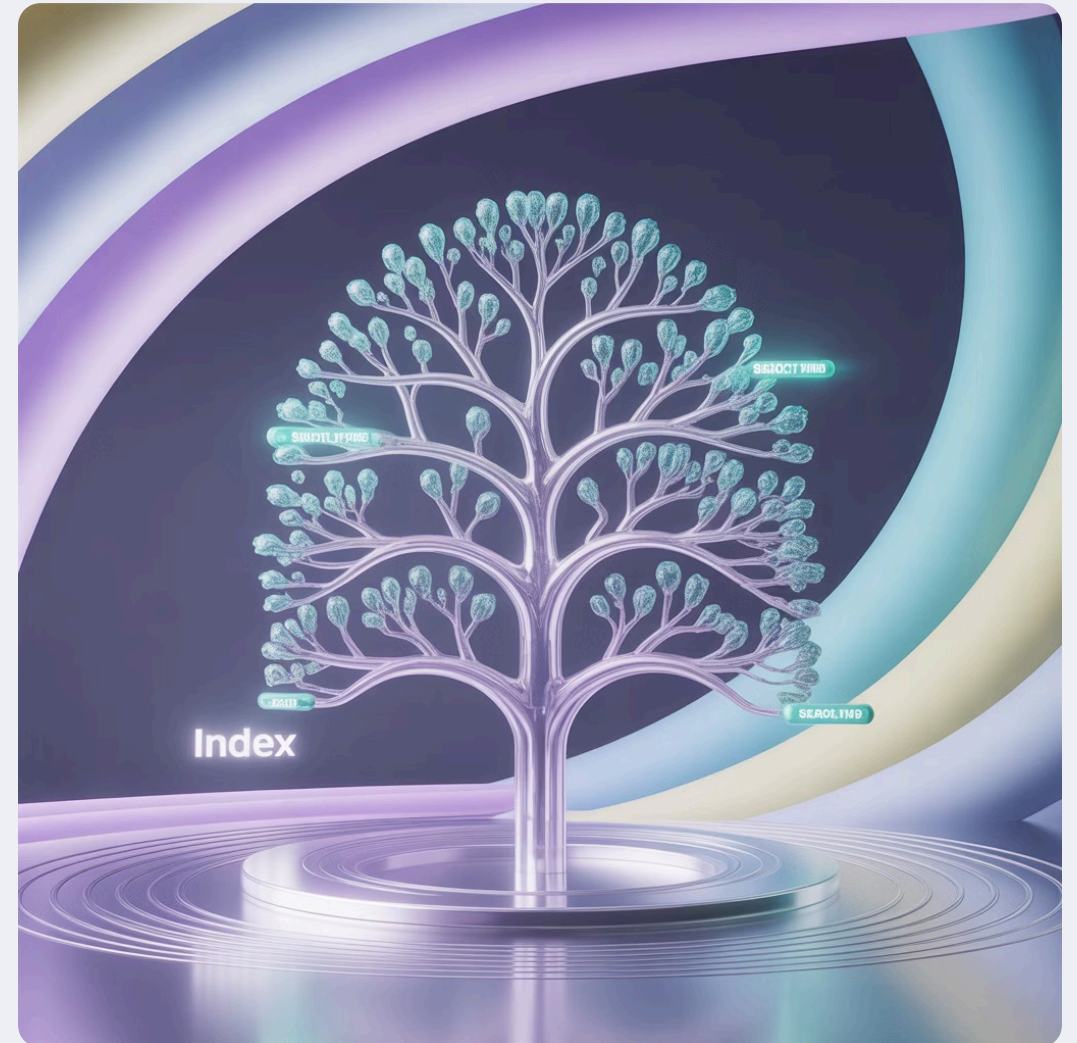
Consulta JOIN

```
SELECT e.Nombre, c.NombreCurso
FROM Estudiantes e
JOIN Matriculas m ON e.Id = m.Estudianteld
JOIN Cursos c ON m.Cursold = c.Id;
```

Optimización con Índices

Los índices son estructuras de datos que mejoran la velocidad de las operaciones en una tabla. Funcionan de manera similar a un índice en un libro, permitiendo al motor de base de datos encontrar rápidamente los registros sin tener que escanear toda la tabla.

```
CREATE INDEX idx_Edad  
ON Estudiantes (Edad);
```



Conclusión y Referencias

Las bases de datos relacionales son esenciales para aplicaciones que gestionan grandes volúmenes de datos estructurados. Esta unidad capacita al estudiante para modelar tablas, establecer relaciones, realizar consultas simples y avanzadas, e integrar la base de datos con sus aplicaciones.

Referencias (APA 7)

- MARK J. PRICE. (2021). C# 10 and .NET 6 Modern Cross-Platform Development.
- Márquez, F. (2015). Unix: programación avanzada. RA-MA.
- Rincón Cardona, J. J. (2025). Aprende desarrollo web con Laravel desde cero.
- Corbí Bellot, A. (2001). Fundamentos de programación.

INFORMACIÓN GENERAL

Fecha de creación	17-07-2025	Dirección:	DIRECCIÓN DE OPERACIONES
-------------------	------------	------------	--------------------------

Escuela	INGENIERIAS
Programa	INGENIERIA DE SISTEMAS

IDENTIFICACIÓN DEL CURSO

Malla:	VG002
Código:	ISP01
Asignatura:	PROGRAMACION AVANZADA
Semestre (Nivel)	4
Temática:	6

Tema: Fundamentos de Programación Web en el Contexto de Aplicaciones

1. Definición

La programación web se refiere al desarrollo de aplicaciones que funcionan en entornos cliente-servidor a través de un navegador web. Esta unidad introduce los principios básicos para estructurar aplicaciones web sencillas desde el punto de vista del backend y el frontend, configurando proyectos básicos en entornos como Visual Studio, manejando formularios y conectándolos a bases de datos para persistencia de datos.

Comprender estos fundamentos permite a los estudiantes diseñar aplicaciones que sean accesibles desde cualquier dispositivo conectado a Internet y que integren múltiples capas de funcionamiento.

2. Conceptos clave

- Estructura básica de aplicaciones web: Incluye la separación de responsabilidades en capas: presentación (HTML, CSS, JavaScript), lógica de negocio (servidor) y persistencia (base de datos).

- Configuración inicial en Visual Studio: Creación de un proyecto web, selección del lenguaje y plantilla adecuada (.NET MVC, WebForms, Razor Pages, etc.) y configuración básica del entorno.
- Manejo de formularios web: Procesamiento de datos enviados por el usuario desde la interfaz, validación en el servidor y respuesta al cliente.
- Conexión básica a bases de datos: Vinculación del backend de la aplicación con una base de datos para almacenar, consultar y actualizar información.
- Pruebas locales y despliegue inicial: Ejecución de la aplicación en un servidor local para pruebas antes de publicarla en un entorno de producción}.

3. Ejemplos en texto

- Ejemplo 1: Formulario simple para capturar datos

```
<form method="post">

    <label>Nombre:</label>

    <input type="text" name="Nombre" />

    <button type="submit">Enviar</button>

</form>
```

Controlador:

```
[HttpPost]

public IActionResult Index(string Nombre)

{

    ViewBag.Mensaje = $"Bienvenido, {Nombre}";

    return View();

}
```

- Ejemplo 2: Conexión a base de datos en .NET Core

```
"ConnectionStrings": {

    "DefaultConnection":

    "Server=localhost;Database=MiBase;Trusted_Connection=True;"

}
```

Luego configurarla en Startup.cs:

```
services.AddDbContext<ApplicationDbContext>(options =>
```

```
options.UseSqlServer(Configuration.GetConnectionString("DefaultConnection"));
```

- Ejemplo 3: Publicación local

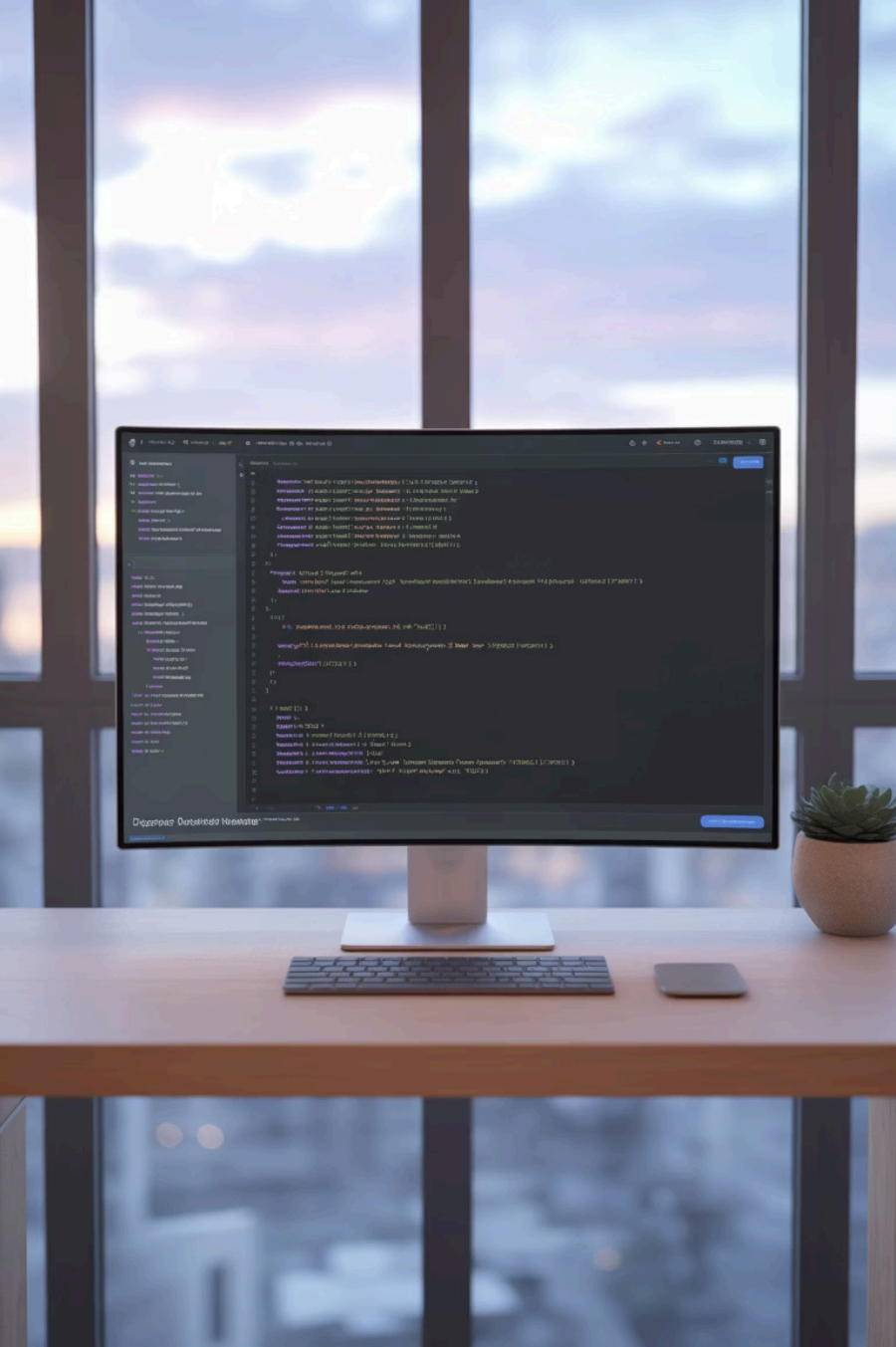
- En Visual Studio: Menú Compilar > Publicar > Carpeta > Elegir ruta.
- Genera los archivos listos para copiar a un servidor web o IIS.

4. Conclusión

Los fundamentos de la programación web son la base para el desarrollo de aplicaciones modernas accesibles a través de navegadores, que interactúan con usuarios y gestionan datos en tiempo real. Dominar la configuración de proyectos, el manejo de formularios, la conexión a bases de datos y el despliegue local son pasos fundamentales para progresar hacia aplicaciones web completas y profesionales.

5. Referencias (APA 7)

- MARK J. PRICE. (2021). C# 10 and .NET 6 – Modern Cross-Platform Development: Build Apps, Websites, and Services with ASP.NET Core 6, Blazor, and EF Core 6 Using Visual Studio 2022 and Visual Studio Code. Packt Publishing.
- Rincón Cardona, J. J. (2025). Aprenda desarrollo web con Laravel desde cero. Marcombo. <https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/281767>
- Contreras Castañeda, M. Á. (2016). Desarrollo de aplicaciones web multiplataforma. Ministerio de Educación y Formación Profesional de España. <https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/101940>



Fundamentos de Programación Web en el Contexto de Aplicaciones

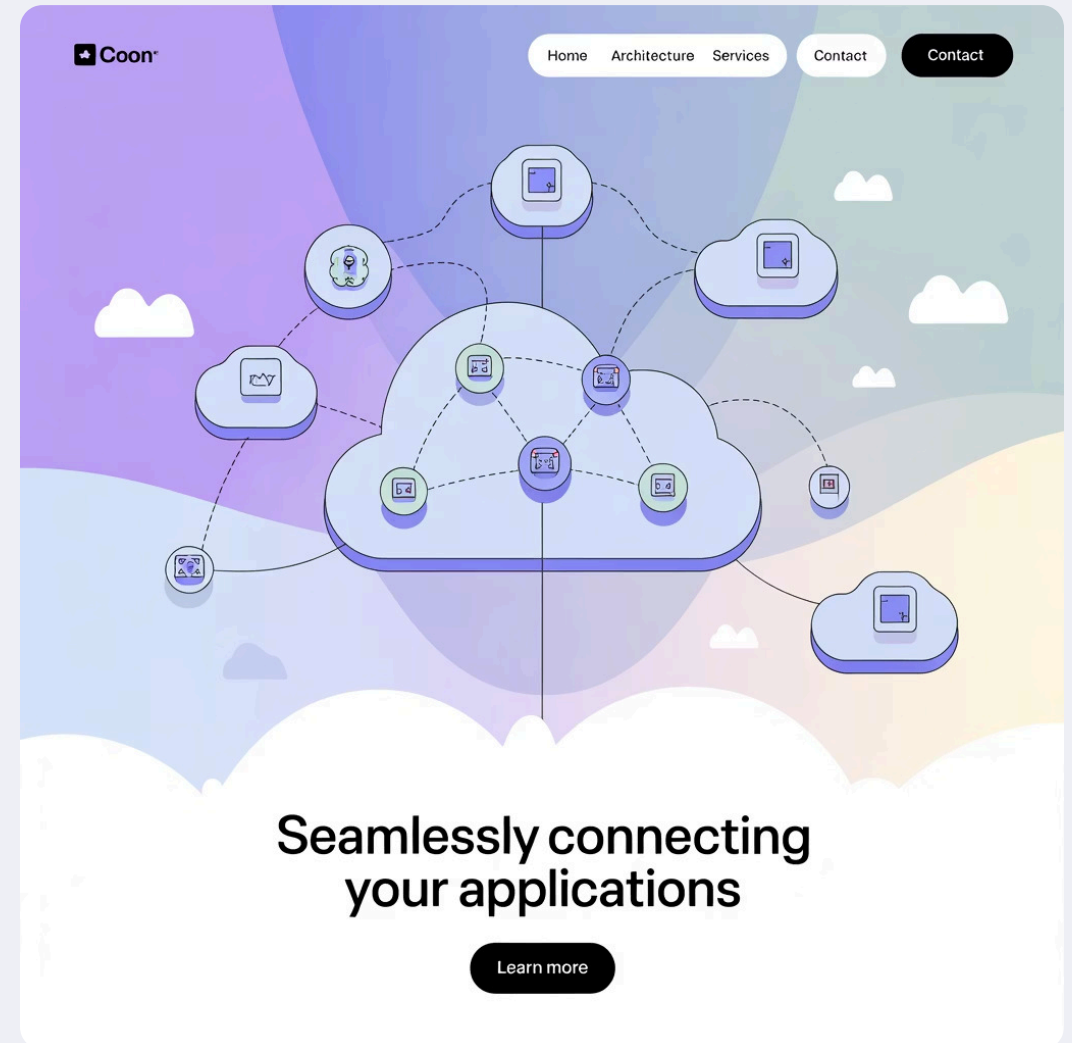
Asignatura: PROGRAMACION AVANZADA

Esta presentación introduce los principios básicos para estructurar aplicaciones web desde el punto de vista del backend y frontend, configurando proyectos en entornos como Visual Studio, manejando formularios y conectándolos a bases de datos para la persistencia de datos.

Definición y Alcance

La programación web se refiere al desarrollo de aplicaciones que funcionan en entornos cliente-servidor a través de un navegador web. Comprender estos fundamentos permite diseñar aplicaciones accesibles desde cualquier dispositivo conectado a Internet e integrar múltiples capas de funcionamiento.

Esta unidad introduce los principios básicos para estructurar aplicaciones web sencillas, configurando proyectos en entornos como Visual Studio, manejando formularios y conectándolos a bases de datos para persistencia de datos.



Las aplicaciones web modernas integran múltiples capas de funcionamiento y permiten el acceso desde cualquier dispositivo conectado a Internet, facilitando la interacción con usuarios y la gestión de datos en tiempo real.

Conceptos Clave en Programación Web

1

Estructura en Capas

Separación de responsabilidades en capas: presentación (HTML, CSS, JavaScript), lógica de negocio (servidor) y persistencia (base de datos).

2

Configuración en Visual Studio

Creación de un proyecto web, selección del lenguaje y plantilla adecuada (.NET MVC, WebForms, Razor Pages, etc.) y configuración básica del entorno.

3

Manejo de Formularios

Procesamiento de datos enviados por el usuario desde la interfaz, validación en el servidor y respuesta al cliente.

4

Conexión a Bases de Datos

Vinculación del backend de la aplicación con una base de datos para almacenar, consultar y actualizar información.

5

Pruebas y Despliegue

Ejecución de la aplicación en un servidor local para pruebas antes de publicarla en un entorno de producción.

Formularios Web: Captura y Procesamiento de Datos

Estructura HTML del Formulario

```
<label>Nombre:</label>
<input type="text" name="Nombre" />
<button
type="submit">Enviar</button>
```

Los formularios son la principal forma de interacción con el usuario en aplicaciones web. Permiten capturar datos que serán procesados por el servidor.

Controlador en el Servidor

```
[HttpPost]
public IActionResult Index(string
Nombre)
{
    ViewBag.Mensaje = $"Bienvenido,
{Nombre}";
    return View();
}
```

El controlador recibe los datos enviados desde el formulario, los procesa según la lógica de negocio y devuelve una respuesta al cliente, que puede ser una vista actualizada o una redirección.

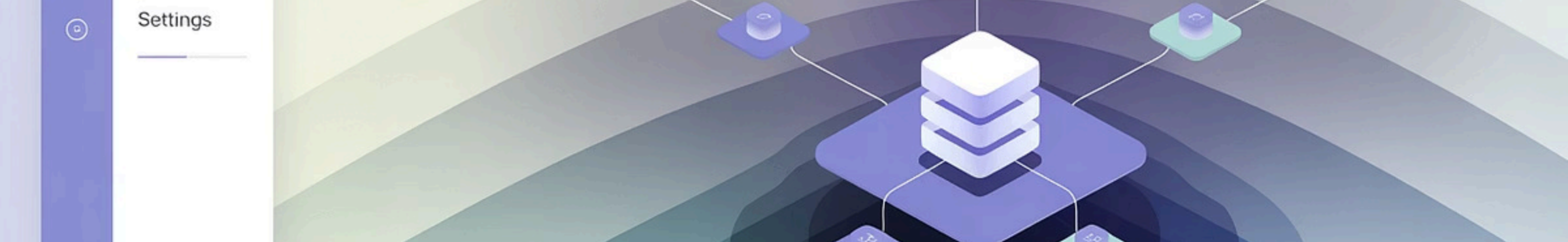
Sign up for our newsletter

Subscribe

 **We respect your privacy**

[Terms of Service](#)

[Privacy Policy](#)



Conexión a Bases de Datos

Configuración de Cadena de Conexión

```
"ConnectionStrings": {  
  "DefaultConnection": "Server=localhost;  
  Database=Mibase;Trusted_Connection=True;"  
}
```

La cadena de conexión define los parámetros necesarios para establecer comunicación con la base de datos, incluyendo servidor, nombre de la base de datos y credenciales.

Configuración en Startup.cs

```
services.AddDbContext(options =>  
  options.UseSqlServer(  
    Configuration.GetConnectionString(  
      "DefaultConnection")));
```

En aplicaciones .NET Core, se configura el contexto de base de datos en el archivo Startup.cs, inyectando el servicio en el contenedor de dependencias.

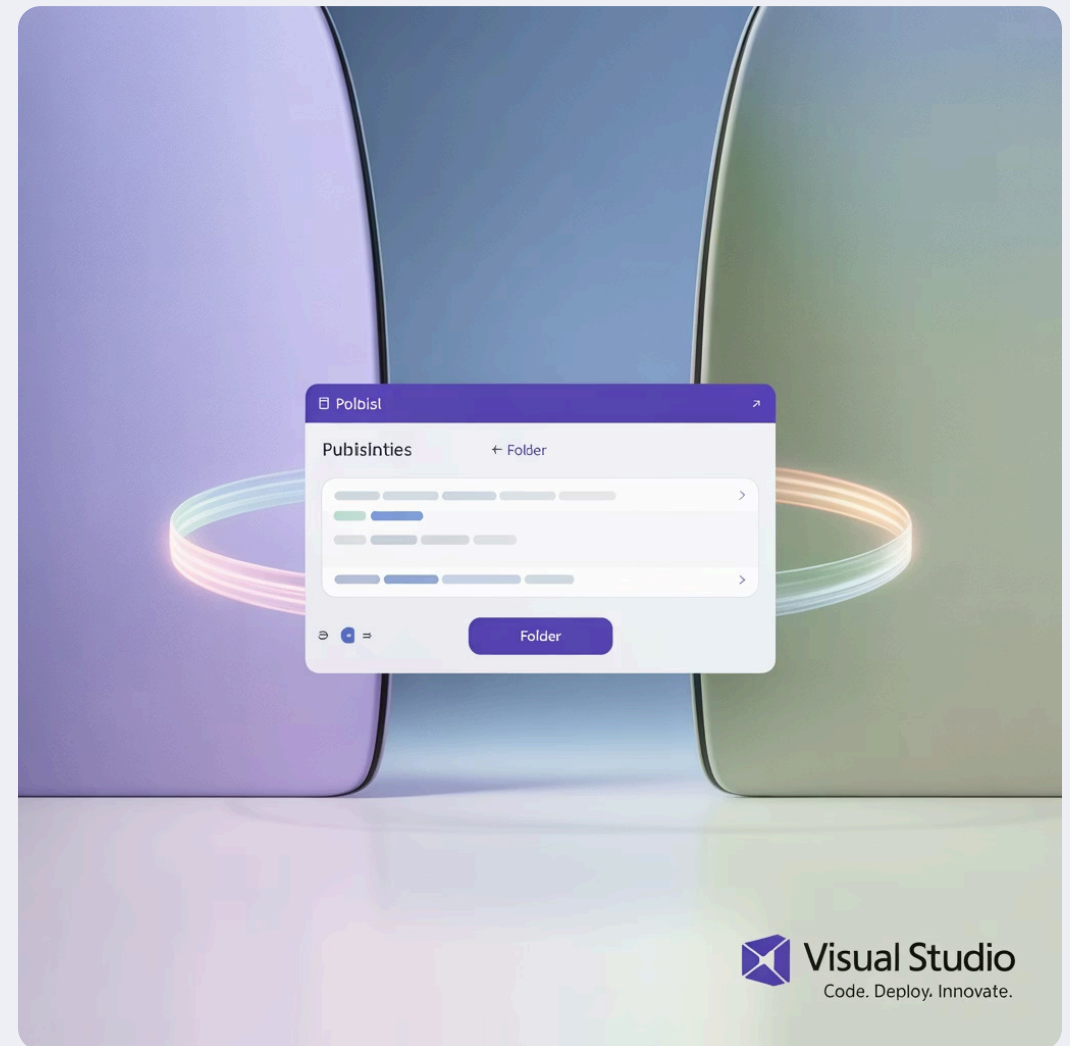
La conexión a bases de datos es fundamental para la persistencia de información en aplicaciones web. Permite almacenar datos de usuarios, productos, transacciones y cualquier otra información relevante para el funcionamiento de la aplicación.

Publicación y Despliegue Local

Proceso de Publicación en Visual Studio

1. Acceder al menú Compilar > Publicar
2. Seleccionar "Carpeta" como destino
3. Elegir ruta de destino en el sistema
4. Configurar parámetros de publicación
5. Iniciar el proceso de publicación

Este proceso genera los archivos listos para copiar a un servidor web o IIS, permitiendo probar la aplicación en un entorno similar al de producción.



El despliegue local es un paso crucial antes de publicar una aplicación en un entorno de producción. Permite detectar y corregir errores en un entorno controlado, asegurando que la aplicación funcione correctamente cuando sea accesible para los usuarios finales.

Conclusiones

Los fundamentos de la programación web son la base para el desarrollo de aplicaciones modernas accesibles a través de navegadores, que interactúan con usuarios y gestionan datos en tiempo real.

Configuración de Proyectos

Establecer correctamente el entorno de desarrollo y la estructura del proyecto es el primer paso para crear aplicaciones web robustas.

Conexión a Bases de Datos

La persistencia de datos permite crear aplicaciones que mantienen estado y ofrecen experiencias personalizadas a los usuarios.

Manejo de Formularios

Dominar la captura, validación y procesamiento de datos de usuario es esencial para la interactividad de las aplicaciones web.

Despliegue y Publicación

El proceso de hacer accesible la aplicación a los usuarios finales completa el ciclo de desarrollo web básico.

Dominar estos conceptos fundamentales es el paso inicial para progresar hacia el desarrollo de aplicaciones web completas y profesionales.

Referencias Bibliográficas

MARK J. PRICE. (2021). **C# 10 and .NET 6 Modern Cross-Platform Development: Build Apps, Websites, and Services with ASP.NET Core 6, Blazor, and EF Core 6 Using Visual Studio 2022 and Visual Studio Code.** Packt Publishing.

Rincón Cardona, J. J. (2025). **Aprenda desarrollo web con Laravel desde cero.** Marcombo.
<https://elibronet.zproxy.cun.edu.co/es/lc/bibliocun/titulos/281767>

Contreras Castañeda, M. Á. (2016). **Desarrollo de aplicaciones web multiplataforma.** Ministerio de Educación y Formación Profesional de España.
<https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/101940>



INFORMACIÓN GENERAL

Fecha de creación	17-07-2025	Dirección:	DIRECCIÓN DE OPERACIONES
-------------------	------------	------------	--------------------------

Escuela	INGENIERIAS
Programa	INGENIERIA DE SISTEMAS

IDENTIFICACIÓN DEL CURSO

Malla:	VG002
Código:	ISP01
Asignatura:	PROGRAMACION AVANZADA
Semestre (Nivel)	4
Temática:	7

Tema: Conceptos de Seguridad en Aplicaciones Básicas

1. Definición

La seguridad en aplicaciones es el conjunto de prácticas, técnicas y mecanismos diseñados para proteger las aplicaciones y los datos que procesan contra accesos no autorizados, manipulación, pérdidas o ataques maliciosos. En esta unidad se abordan las prácticas esenciales para proteger aplicaciones básicas, como la validación de entradas, encriptación de datos y autenticación de usuarios.

Incorporar estos principios desde las primeras etapas del desarrollo permite crear aplicaciones más robustas y minimizar vulnerabilidades comunes.

2. Conceptos clave

- Buenas prácticas para proteger datos: Incluyen medidas como limitar el acceso a información sensible, utilizar conexiones seguras y evitar el almacenamiento de contraseñas en texto plano.
- Validación de entradas: Verificar los datos introducidos por los usuarios para asegurarse de que cumplen las condiciones esperadas, evitando ataques como SQL Injection o Cross-site scripting (XSS).

- Encriptación básica de datos sensibles: Transformar los datos confidenciales a un formato ilegible para terceros no autorizados, usando algoritmos como SHA256 para contraseñas o AES para datos.
 - Autenticación básica: Proceso mediante el cual se verifica la identidad de un usuario antes de permitirle el acceso a una aplicación o sus funcionalidades.
-

3. Ejemplos en texto

- Ejemplo 1: Validación de entradas en C# Verifica que un campo no contenga caracteres no permitidos:

```
if(!Regex.IsMatch(usuarioInput, @"^[a-zA-Z0-9]+$"))  
  
{  
  
    MessageBox.Show("El nombre de usuario solo puede contener letras y  
    números.");  
  
    return;  
  
}
```

- Ejemplo 2: Encriptación de contraseñas con SHA256

```
using System.Security.Cryptography;  
  
using System.Text;  
  
public string EncriptarSHA256(string texto)  
{  
  
    using(SHA256 sha256 = SHA256.Create())  
  
    {  
  
        byte[] bytes = sha256.ComputeHash(Encoding.UTF8.GetBytes(texto));  
  
        StringBuilder builder = new StringBuilder();  
  
        foreach(var b in bytes)  
  
        {  
  
            builder.Append(b.ToString("x2"));  
  
        }  
  
        return builder.ToString();  
  
    }  
  
}
```

- Ejemplo 3: Autenticación básica. Comparar la contraseña ingresada con la almacenada:

```
string contraseñaIngresada = txtPassword.Text;

string contraseñaEncriptada = EncriptarSHA256(contraseñaIngresada);

if(contraseñaEncriptada == contraseñaGuardada)

{

    MessageBox.Show("Acceso concedido.");

}

else

{

    MessageBox.Show("Credenciales inválidas.");

}
```

4. Conclusión

Incorporar prácticas básicas de seguridad desde el inicio del desarrollo es fundamental para proteger la integridad de las aplicaciones y los datos de los usuarios. Validar las entradas, encriptar datos sensibles y autenticar usuarios correctamente contribuye a construir confianza en las aplicaciones y mitigar riesgos comunes. Esta unidad proporciona las herramientas iniciales para que los estudiantes desarrollen aplicaciones más seguras y profesionales.

5. Referencias (APA 7)

- Hernández Bejarno, M. (2020). Ciclo de vida de desarrollo ágil de software seguro. Fundación Universitaria Los Libertadores. <https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/197008>
- Moreno García, M. (2022). Gestión de incidentes de ciberseguridad: programación avanzada. Catálogo Digital CUN.
- MARK J. PRICE. (2021). C# 10 and .NET 6 – Modern Cross-Platform Development: Build Apps, Websites, and Services with ASP.NET Core 6, Blazor, and EF Core 6 Using Visual Studio 2022 and Visual Studio Code. Packt Publishing.



Conceptos de Seguridad en Aplicaciones Básicas

Bienvenidos a esta presentación sobre seguridad en aplicaciones básicas. Exploraremos los fundamentos esenciales para proteger nuestras aplicaciones y los datos que procesan contra accesos no autorizados, manipulación, pérdidas o ataques maliciosos. Incorporar estos principios desde las primeras etapas del desarrollo nos permitirá crear aplicaciones más robustas y minimizar vulnerabilidades comunes.



Shieldtech

"Secure your digital future"

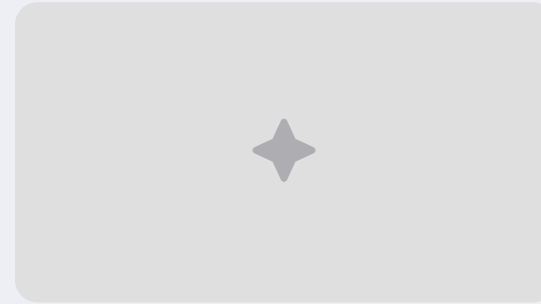
¿Qué es la Seguridad en Aplicaciones?

La seguridad en aplicaciones es el conjunto de prácticas, técnicas y mecanismos diseñados para proteger las aplicaciones y los datos que procesan contra accesos no autorizados, manipulación, pérdidas o ataques maliciosos.

En esta unidad abordaremos las prácticas esenciales para proteger aplicaciones básicas, como:

- Validación de entradas
- Encriptación de datos
- Autenticación de usuarios

Incorporar estos principios desde las primeras etapas del desarrollo permite crear aplicaciones más robustas y minimizar vulnerabilidades comunes que podrían comprometer la integridad de nuestros sistemas.



La seguridad debe considerarse en todas las capas de la aplicación, desde la interfaz de usuario hasta la base de datos, implementando múltiples niveles de protección.

Conceptos Clave de Seguridad

1

Buenas Prácticas para Proteger Datos

Incluyen medidas como limitar el acceso a información sensible, utilizar conexiones seguras y evitar el almacenamiento de contraseñas en texto plano.

- Principio de mínimo privilegio
- Uso de HTTPS para transmisiones
- Almacenamiento seguro de credenciales

2

Validación de Entradas

Verificar los datos introducidos por los usuarios para asegurarse de que cumplen las condiciones esperadas, evitando ataques como SQL Injection o Cross-site scripting (XSS).

- Validación del lado del cliente y servidor
- Uso de expresiones regulares
- Sanitización de datos

3

Encriptación Básica

Transformar los datos confidenciales a un formato ilegible para terceros no autorizados, usando algoritmos como SHA256 para contraseñas o AES para datos.

- Funciones hash para contraseñas
- Cifrado simétrico y asimétrico
- Gestión segura de claves

4

Autenticación Básica

Proceso mediante el cual se verifica la identidad de un usuario antes de permitirle el acceso a una aplicación o sus funcionalidades.

- Sistemas de login seguros
- Autenticación de múltiples factores
- Gestión de sesiones

Validación de Entradas

¿Por qué es importante?

La validación de entradas es la primera línea de defensa contra ataques como inyección SQL, XSS y manipulación de datos. Permite verificar que los datos introducidos por los usuarios cumplen con los formatos y restricciones esperados.

Beneficios clave:

- Previene la ejecución de código malicioso
- Mantiene la integridad de los datos
- Mejora la experiencia del usuario al proporcionar retroalimentación inmediata

Ejemplo en C#

```
if(!Regex.IsMatch(usuarioInput, @"^[a-zA-Z0-9]+$"))
{
    MessageBox.Show("El nombre de usuario solo puede contener letras y números.");
    return;
}
```

Este código verifica que un campo de entrada de usuario contenga únicamente letras y números, rechazando caracteres especiales que podrían utilizarse en ataques.

La expresión regular **^[a-zA-Z0-9]+\$** garantiza que solo se acepten caracteres alfanuméricos, bloqueando cualquier intento de inyección de código malicioso.

Encriptación de Datos Sensibles

Fundamentos de la Encriptación

La encriptación transforma datos legibles en un formato codificado que solo puede ser descifrado con la clave correcta. Es esencial para proteger información sensible como contraseñas, datos personales y financieros.

Tipos principales:

- **Funciones hash:** Transformaciones unidireccionales (SHA256, bcrypt)
- **Cifrado simétrico:** Usa la misma clave para cifrar y descifrar (AES)
- **Cifrado asimétrico:** Utiliza pares de claves pública/privada (RSA)

Ejemplo: Encriptación SHA256 en C#

```
using System.Security.Cryptography;
using System.Text;

public string EncriptarSHA256(string texto)
{
    using(SHA256 sha256 = SHA256.Create())
    {
        byte[] bytes = sha256.ComputeHash(
            Encoding.UTF8.GetBytes(texto));
        StringBuilder builder = new StringBuilder();
        foreach(var b in bytes)
        {
            builder.Append(b.ToString("x2"));
        }
        return builder.ToString();
    }
}
```

Este método convierte una cadena de texto en un hash SHA256 irreversible, ideal para almacenar contraseñas de forma segura. El resultado es una cadena hexadecimal de 64 caracteres que no puede ser revertida al texto original.

Autenticación de Usuarios

Recepción de Credenciales

El sistema recibe el nombre de usuario y contraseña ingresados por el usuario a través de un formulario seguro.

Es importante utilizar conexiones HTTPS para proteger la transmisión de estas credenciales y evitar ataques de intermediarios.

Verificación de Credenciales

Se compara el hash generado con el almacenado en la base de datos para determinar si las credenciales son correctas.

```
string contraseñaIngresada = txtPassword.Text;
string contraseñaEncriptada =
    EncriptarSHA256(contraseñaIngresada);
if(contraseñaEncriptada == contraseñaGuardada) {
    MessageBox.Show("Acceso concedido.");
} else {
    MessageBox.Show("Credenciales inválidas.");
}
```

Procesamiento de la Contraseña

La contraseña ingresada se procesa utilizando el mismo algoritmo de hash que se utilizó al registrar al usuario.

Nunca se debe comparar contraseñas en texto plano ni almacenarlas sin encriptar en la base de datos.

Gestión de Sesión

Tras una autenticación exitosa, se establece una sesión segura utilizando tokens o cookies con medidas de protección adecuadas.

Es recomendable implementar tiempos de expiración y renovación de sesiones para minimizar riesgos.

Implementación Práctica de Seguridad

Validación Completa

Implementar validación tanto en el cliente (JavaScript) como en el servidor (C#) para garantizar que todos los datos cumplen con los requisitos establecidos.

- Validar tipo de datos (numérico, texto, fecha)
- Verificar rangos y longitudes permitidas
- Comprobar formatos específicos (email, teléfono)

Almacenamiento Seguro

Utilizar técnicas adecuadas para el almacenamiento de información sensible, especialmente credenciales de usuarios.

- Usar funciones hash con sal para contraseñas
- Cifrar datos personales sensibles
- Implementar control de acceso a nivel de base de datos

Gestión de Errores

Implementar un sistema de gestión de errores que no revele información sensible sobre la aplicación o su infraestructura.

- Mostrar mensajes genéricos al usuario
- Registrar detalles técnicos en logs seguros
- Notificar a administradores sobre errores críticos

Caso Práctico: Formulario de Registro

Un formulario de registro seguro debe implementar:

1. Validación de complejidad de contraseñas
2. Verificación de unicidad de nombre de usuario/email
3. Protección contra ataques de fuerza bruta
4. Confirmación por email u otro factor
5. Almacenamiento seguro de credenciales

Create Your Account

[Login](#) [Forgot Password?](#)

Username

Email

Password

weak

fair

good

strong

Coeur Stont

Conclusiones y Referencias

Conclusiones Clave

Incorporar prácticas básicas de seguridad desde el inicio del desarrollo es fundamental para proteger la integridad de las aplicaciones y los datos de los usuarios. Validar las entradas, encriptar datos sensibles y autenticar usuarios correctamente contribuye a construir confianza en las aplicaciones y mitigar riesgos comunes.

Esta unidad proporciona las herramientas iniciales para que los estudiantes desarrollen aplicaciones más seguras y profesionales, estableciendo una base sólida para prácticas de seguridad más avanzadas en el futuro.

 Recuerda: La seguridad no es un producto, sino un proceso continuo que requiere atención constante y actualización de conocimientos.

Referencias (APA 7)

- Hernández Bejarno, M. (2020). *Ciclo de vida de desarrollo ágil de software seguro*. Fundación Universitaria Los Libertadores.
<https://elibronet.zproxy.cun.edu.co/es/lc/bibliocun/titulos/197008>
- Moreno García, M. (2022). *Gestión de incidentes de ciberseguridad: programación avanzada*. Catálogo Digital CUN.
- Price, M. J. (2021). *C# 10 and .NET 6 Modern Cross-Platform Development: Build Apps, Websites, and Services with ASP.NET Core 6, Blazor, and EF Core 6 Using Visual Studio 2022 and Visual Studio Code*. Packt Publishing.

Para más información sobre seguridad en aplicaciones, consulte los recursos oficiales de OWASP (Open Web Application Security Project).

INFORMACIÓN GENERAL

Fecha de creación	17-07-2025	Dirección:	DIRECCIÓN DE OPERACIONES
-------------------	------------	------------	--------------------------

Escuela	INGENIERIAS
Programa	INGENIERIA DE SISTEMAS

IDENTIFICACIÓN DEL CURSO

Malla:	VG002
Código:	ISP01
Asignatura:	PROGRAMACION AVANZADA
Semestre (Nivel)	4
Temática:	8

Tema: Automatización de Tareas con Scripts Básicos

1. Definición

La automatización de tareas consiste en escribir pequeños programas o scripts que permiten ejecutar procesos repetitivos de forma automática y eficiente, reduciendo errores humanos y mejorando la productividad. En esta unidad, el estudiante aprenderá a utilizar sentencias básicas para automatizar tareas comunes como generación de reportes, manipulación de archivos y procesamiento de datos.

Los scripts son especialmente útiles para tareas administrativas, pruebas automáticas, mantenimiento de sistemas y procesos programados.

2. Conceptos clave

- Uso de scripts para tareas repetitivas: Crear programas pequeños para ejecutar acciones rutinarias como copias de seguridad, limpieza de archivos temporales o carga de datos.
- Automatización de procesos comunes: Diseñar scripts para procesos típicos en aplicaciones empresariales, como la generación de reportes periódicos o la actualización de registros.

- Manejo de archivos: Leer, escribir y procesar datos almacenados en archivos de texto, CSV o similares.
 - Procesamiento de datos simples: Aplicar lógica básica para transformar, filtrar o resumir información a partir de archivos o entradas.
-

3. Ejemplos en texto

- Ejemplo 1: Crear un archivo de log con la fecha actual:

```
string ruta = @"C:\logs\";

string nombreArchivo = $"log_{DateTime.Now:yyyyMMdd}.txt";

File.WriteAllText(Path.Combine(ruta, nombreArchivo), "Proceso
completado.");
```

- Ejemplo 2: Leer y mostrar líneas de un archivo CSV

```
string[] lineas = File.ReadAllLines(@"C:\datos\estudiantes.csv");

foreach(string linea in lineas)

{

    string[] campos = linea.Split(',');

    Console.WriteLine($"Nombre: {campos[0]}, Edad: {campos[1]}");

}
```

- Ejemplo 3: Generar reporte automático de ventas

```
List<string> ventas = new List<string>()

{

    "Producto A,10,5000",

    "Producto B,5,3000"

};

File.WriteAllLines(@"C:\reportes\ventas.txt", ventas);

Console.WriteLine("Reporte generado exitosamente.");
```

- Ejemplo 4: Eliminar archivos temporales

```
string[] archivos = Directory.GetFiles(@"C:\temp\", "*.tmp");

foreach(string archivo in archivos)

{

    File.Delete(archivo);

}
```

}

```
Console.WriteLine("Archivos temporales eliminados.");
```

4. Conclusión

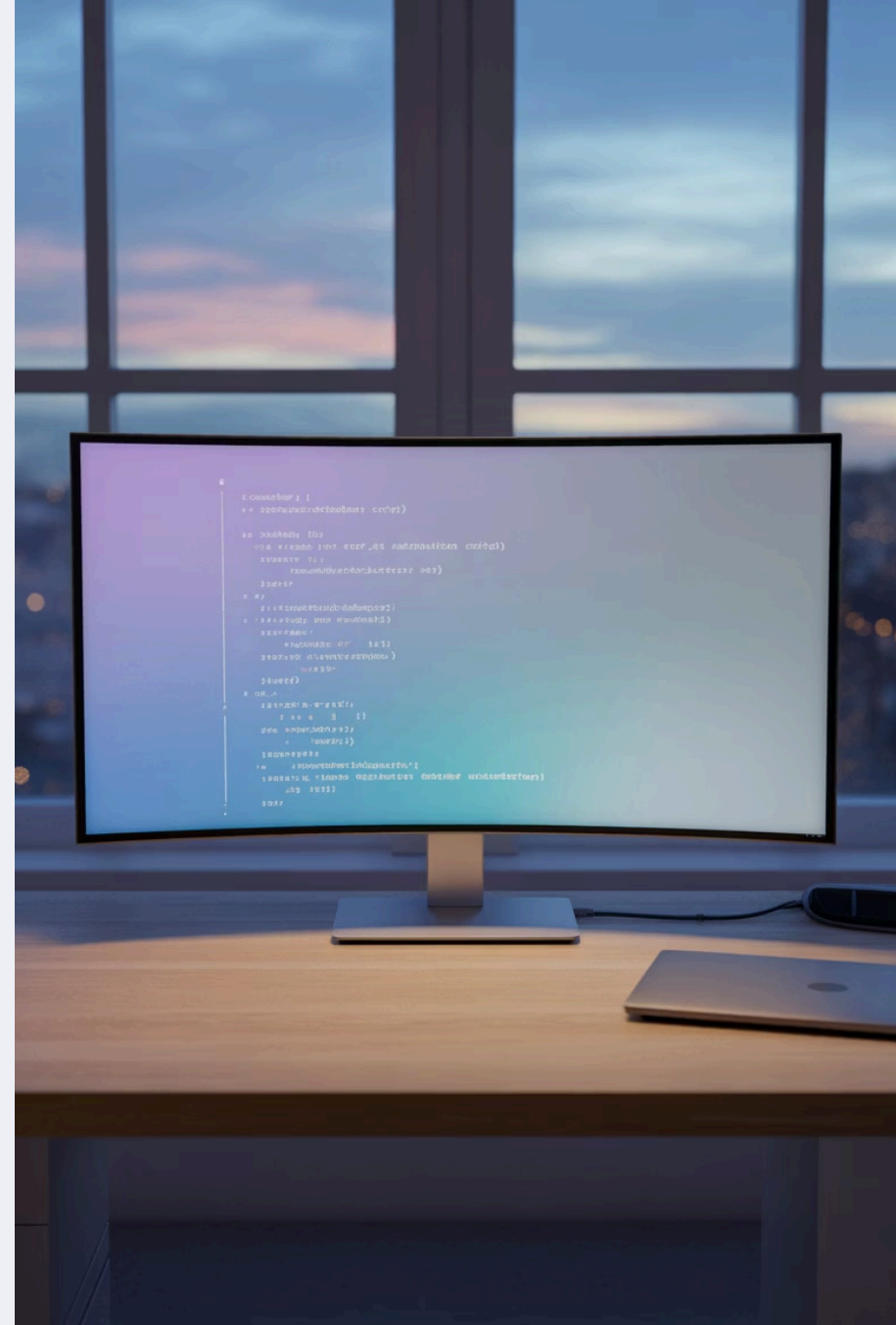
Automatizar tareas mediante scripts básicos permite ahorrar tiempo, estandarizar procesos y minimizar errores en tareas repetitivas. Los estudiantes desarrollarán la capacidad de diseñar soluciones rápidas y eficientes para problemas cotidianos, consolidando una habilidad esencial para el mantenimiento y la administración de sistemas y datos.

5. Referencias (APA 7)

- Luna Fernández, A., & Sanz Mercado, P. (2014). Programación de Shell Scripts. Editorial Universidad Autónoma de Madrid. <https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/53965>
- Márquez García, F. M. (2014). UNIX: programación avanzada. Catálogo Digital CUN.
- MARK J. PRICE. (2021). C# 10 and .NET 6 – Modern Cross-Platform Development: Build Apps, Websites, and Services with ASP.NET Core 6, Blazor, and EF Core 6 Using Visual Studio 2022 and Visual Studio Code. Packt Publishing.

Automatización de Tareas con Scripts Básicos

Bienvenidos a esta presentación sobre automatización de tareas mediante scripts básicos. Exploraremos cómo estos pequeños programas pueden transformar procesos repetitivos en soluciones eficientes, ahorrando tiempo y reduciendo errores humanos. A lo largo de esta presentación, descubriremos conceptos clave, ejemplos prácticos y aplicaciones reales que demuestran el poder de la automatización en el desarrollo de software moderno.

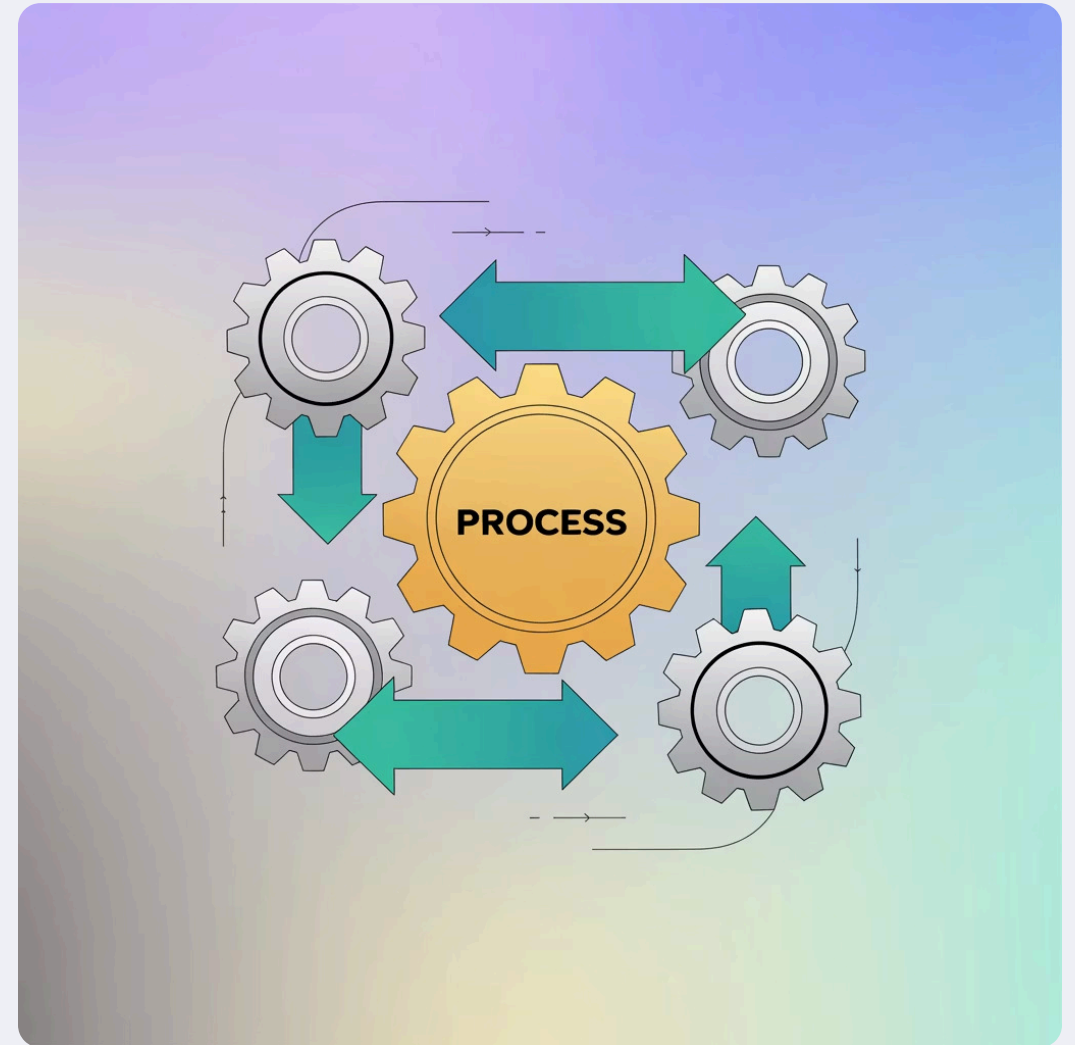


¿Qué es la Automatización de Tareas?

La automatización de tareas consiste en escribir pequeños programas o scripts que permiten ejecutar procesos repetitivos de forma automática y eficiente, reduciendo errores humanos y mejorando la productividad. Esta técnica es fundamental para optimizar flujos de trabajo en entornos de desarrollo y administración de sistemas.

Los scripts son especialmente útiles para:

- Tareas administrativas rutinarias
- Pruebas automáticas de software
- Mantenimiento de sistemas
- Procesos programados periódicos
- Generación de informes y reportes



En esta unidad, aprenderemos a utilizar sentencias básicas para automatizar tareas comunes como generación de reportes, manipulación de archivos y procesamiento de datos, sentando las bases para desarrollar soluciones más complejas en el futuro.

Conceptos Clave en Automatización

1

Uso de Scripts para Tareas Repetitivas

Crear programas pequeños para ejecutar acciones rutinarias como copias de seguridad, limpieza de archivos temporales o carga de datos. Estos scripts permiten programar tareas que de otro modo consumirían tiempo valioso del desarrollador.

2

Automatización de Procesos Comunes

Diseñar scripts para procesos típicos en aplicaciones empresariales, como la generación de reportes periódicos o la actualización de registros en bases de datos. Esto garantiza consistencia y puntualidad en la ejecución.

3

Manejo de Archivos

Leer, escribir y procesar datos almacenados en archivos de texto, CSV o similares. Esta capacidad es fundamental para la automatización de tareas que involucran intercambio de información entre sistemas.

4

Procesamiento de Datos Simples

Aplicar lógica básica para transformar, filtrar o resumir información a partir de archivos o entradas. Esto permite convertir datos crudos en información útil para la toma de decisiones.

Ejemplo 1: Creación de Archivos de Log

Descripción

Este ejemplo muestra cómo crear automáticamente un archivo de registro (log) con la fecha actual incorporada en el nombre del archivo. Esta técnica es esencial para sistemas que requieren seguimiento cronológico de eventos.

Beneficios:

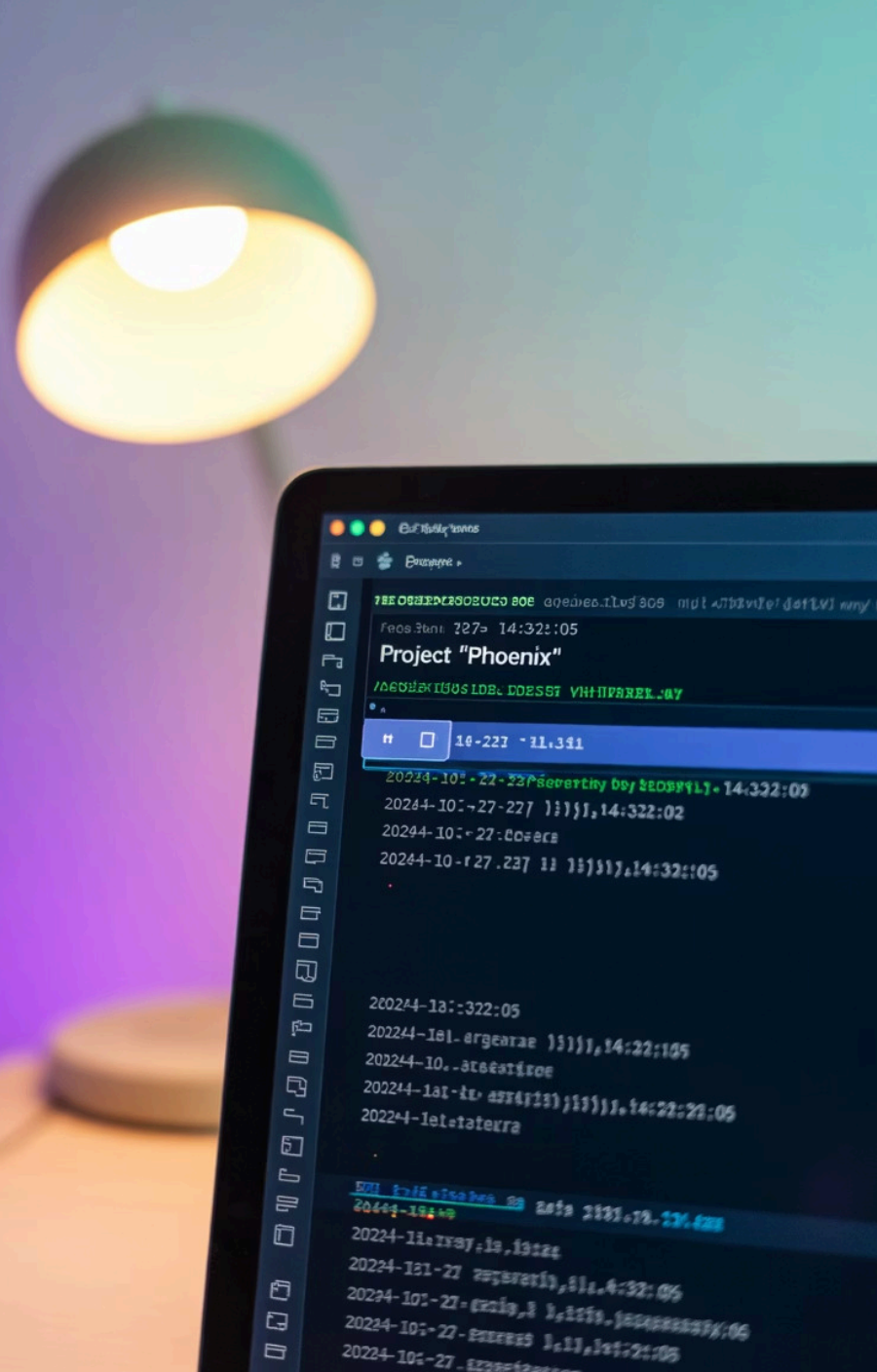
- Organización cronológica de registros
- Facilita la búsqueda de información histórica
- Permite implementar políticas de retención de datos
- Automatiza la documentación de procesos

```
string ruta = @"C:\\logs\\";

string nombreArchivo =
    $"log_{DateTime.Now:yyyyMMdd}.txt";

File.WriteAllText(Path.Combine(ruta,
                                nombreArchivo),
                  "Proceso completado.");
```

El código utiliza la clase **DateTime.Now** para obtener la fecha actual y formatearla como parte del nombre del archivo. La clase **File** proporciona métodos estáticos para trabajar con archivos, mientras que **Path.Combine** asegura la correcta concatenación de rutas independientemente del sistema operativo.



Ejemplo 2: Procesamiento de Archivos CSV

```
string[] lineas = File.ReadAllLines(
    @"C:\datos\lestudiantes.csv");

foreach(string linea in lineas)
{
    string[] campos = linea.Split(',');
    Console.WriteLine(
        $"Nombre: {campos[0]}, Edad: {campos[1]}");
}
```

Este script lee un archivo CSV que contiene información de estudiantes y muestra cada registro en la consola. El método **File.ReadAllLines** carga todo el contenido del archivo en un array de strings, mientras que el método **Split** divide cada línea en sus componentes utilizando la coma como delimitador.

Aplicaciones Prácticas

El procesamiento de archivos CSV es una tarea común en entornos empresariales y educativos, donde se utiliza para:

- Importación de datos desde sistemas externos
- Procesamiento de información de estudiantes o clientes
- Análisis de datos de ventas o inventario
- Migración de información entre plataformas
- Generación de reportes personalizados

La capacidad de procesar estos archivos de forma automática elimina la necesidad de realizar tareas manuales propensas a errores y aumenta significativamente la productividad.

Ejemplos 3 y 4: Reportes y Limpieza

Ejemplo 3: Generación de Reportes

```
List ventas = new List()
{
    "Producto A,10,5000",
    "Producto B,5,3000"
};

File.WriteAllLines(@"C:\\reportes\\ventas.txt", ventas);
Console.WriteLine("Reporte generado exitosamente.");
```

Este script crea un reporte de ventas simple guardando una lista de strings en un archivo de texto. Cada línea representa un registro de venta con el nombre del producto, cantidad y valor, separados por comas.

Ejemplo 4: Limpieza de Archivos Temporales

```
string[] archivos = Directory.GetFiles(
    @"C:\\temp", "*.tmp");

foreach(string archivo in archivos)
{
    File.Delete(archivo);
}

Console.WriteLine("Archivos temporales eliminados.");
```

Este script busca y elimina todos los archivos con extensión .tmp en el directorio temporal. Utiliza **Directory.GetFiles** para obtener la lista de archivos que coinciden con el patrón especificado y luego los elimina uno por uno.

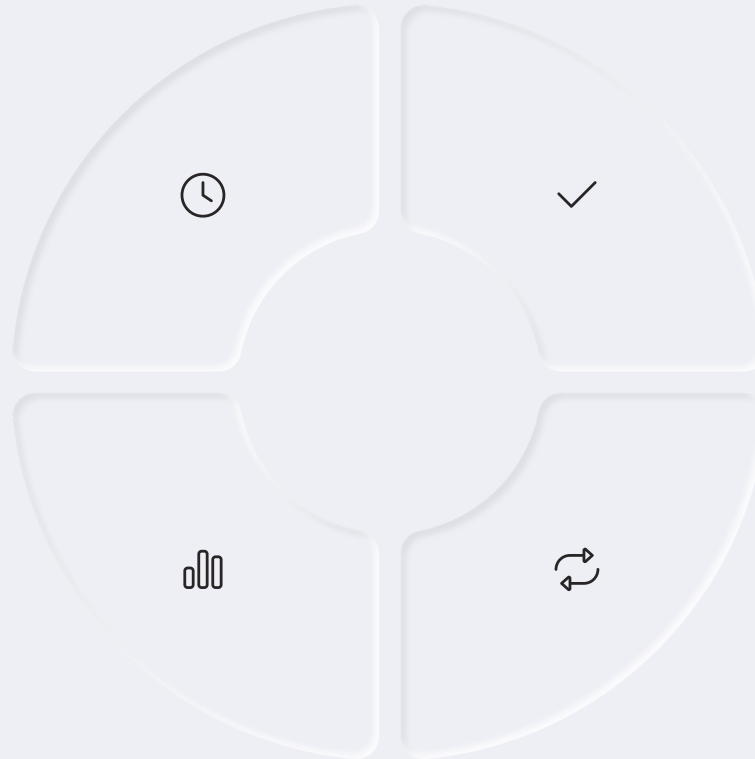
Beneficios de la Automatización

Ahorro de Tiempo

Reduce drásticamente el tiempo dedicado a tareas repetitivas, permitiendo a los desarrolladores enfocarse en actividades de mayor valor.

Escalabilidad

Facilita el manejo de volúmenes crecientes de datos o tareas sin necesidad de recursos humanos adicionales.



Reducción de Errores

Minimiza los errores humanos al estandarizar procesos y eliminar la intervención manual en tareas rutinarias.

Consistencia

Garantiza que los procesos se ejecuten siempre de la misma manera, independientemente de quién o cuándo se inicien.

Automatizar tareas mediante scripts básicos permite ahorrar tiempo, estandarizar procesos y minimizar errores en tareas repetitivas. Los estudiantes desarrollarán la capacidad de diseñar soluciones rápidas y eficientes para problemas cotidianos, consolidando una habilidad esencial para el mantenimiento y la administración de sistemas y datos.

Referencias y Recursos Adicionales

Referencias Bibliográficas (APA 7)

- Luna Fernández, A., & Sanz Mercado, P. (2014). *Programación de Shell Scripts*. Editorial Universidad Autónoma de Madrid.
<https://elibronet.zproxy.cun.edu.co/es/lc/bibliocun/titulos/53965>
- Márquez García, F. M. (2014). *UNIX: programación avanzada*. Catálogo Digital CUN.
- MARK J. PRICE. (2021). *C# 10 and .NET 6 Modern Cross-Platform Development: Build Apps, Websites, and Services with ASP.NET Core 6, Blazor, and EF Core 6 Using Visual Studio 2022 and Visual Studio Code*. Packt Publishing.

Próximos Pasos

Para profundizar en la automatización de tareas, se recomienda:

- Explorar bibliotecas específicas para manipulación de archivos y datos
- Aprender sobre programación de tareas con el Programador de Tareas de Windows o Cron en sistemas Unix
- Estudiar técnicas de manejo de errores y logging para scripts robustos
- Investigar herramientas de automatización más avanzadas como PowerShell o Python

La automatización es una habilidad fundamental que seguirá siendo relevante a lo largo de toda su carrera profesional en programación.